Randomized Kernel Ridge Regression - Report

Ebe-Jan Buitenhuis (s4158628) Caolan Leonard (s6156681) Jesse van der Zeijden (s4116801)

April 13, 2026

1 Introduction

Kernel Ridge Regression (KRR) provides a powerful framework for non-linear regression, which can rephrase minimization problems in terms of solving a single linear system. This system is however of the size $n \times n$, where n is the number of data points and in real-life minimization settings, this can be a rather large number. This has a major effect on the computational cost of solving the KRR minimization problem, which might make it an unfeasible approach in practice.

In this report we set out to investigate whether the use of randomized techniques (and Nyström approximations in particular) can be of use in this problem setting. Ideally we would like to find out whether a randomized approach can solve these types of minimization problems with the same accuracy as a classical approach, but at a considerably lower computational cost.

In section 2 of this report, we start by deriving the KRR minimization problem and how it reduces to solving a linear system. Afterwards, in section 3, we introduce the classical approach of solving the resulting linear system and apply it to three chosen regression-type data sets. These experiments will reveal how we are hindered by a specific computational bottleneck: accurate KRR results require the explicit formation of the entire (so-called) kernel matrix $K \in \mathbb{R}^{n \times n}$, which becomes impractically expensive for larger n. The reason for this, is the fact that each entry of K is the result of the evaluation of a computationally expensive function.

This conclusion will motivate section 4 onwards, where we will first delve into the low-rank nature of the kernel matrices, allowing for low-rank approximation techniques to be implemented as a solution to the aforementioned bottleneck. Sections 5 and 6 will utilise this low-rank structure, by introducing the *Nyström approximation*, $\hat{K}$, of the kernel matrix. Central to this idea will be the sampling of columns of the kernel matrix in order to construct our kernel matrix approximation. To this end, different approaches to sampling these columns will be studied in section 6.

All plots in this report were generated using the Python scripts and data sets that are included in the separate appendix.

2 The Kernel Ridge Regression (KRR) Problem

The webpage by R. Zhu [Zhu, 2025] has been the main inspiration and source for this section.

Problem setting

The focus of this report is on a specific class of optimisation problems, namely those concerned with finding an approximation to some hidden/unknown function $g: \mathbb{R}^m \to \mathbb{R}$. This approximation is made based on n available (training) data points, which each consist of an input $x_i \in \mathbb{R}^m$ and an output $y_i \in \mathbb{R}$. These inputs and outputs are sometimes stacked together into a matrix $X(\in \mathbb{R}^{m\times n})$ and a vector $Y(\in \mathbb{R}^n)$, respectively.

A common approach to approximating this function g, is to minimize the average Euclidean error over all possible functions. More formally, we aim to find a function $\hat{f}$ such that

$$\hat{f} = \underset{f\in\mathcal{F}}{\operatorname{argmin}} \frac{1}{n}\|f(x) - y\|_2^2 = \underset{f\in\mathcal{F}}{\operatorname{argmin}} \frac{1}{n}\sum_{i=1}^{n}(f(x_i) - y_i)^2 \tag{1}$$

where (for now) $\mathcal{F}$ is the space of all well-defined functions that map from $\mathbb{R}^m \to \mathbb{R}$.

Linear regression as an example

To get a feel for what we are doing here, we consider the problem of linear regression. Let us assume that the underlying function g admits a somewhat linear structure, which means that the data points exhibit a somewhat linear relationship. If we now restrict $\mathcal{F}$ to contain only the linear functions from $\mathbb{R}^m \to \mathbb{R}$, our minimization problem (1) simply becomes the typical linear regression problem:

$$\hat{f}(\cdot) = \hat{a}(\cdot) + \hat{b} = \underset{a,b\in\mathbb{R}}{\operatorname{argmin}} \sum_{i=1}^{n}((ax_i + b) - y_i)^2$$

Regularisation

The current set-up is very prone to over-fitting, as the minimization only focuses on accurately predicting the training data points, which may result in a very complicated/specialised approximation $\hat{f}$. This might perform really well when predicting the training data points, but if the approximation $\hat{f}$ is way more complex than the underlying function g is, then its predictions will be poor when presented with new, unseen (testing) data points. Readers familiar with the fields of data science and statistical/machine learning will recognize this as the *Bias-Variance Trade-off* (see figure 1).

A common approach to addressing this issue, is to add a regularisation term and the resulting problem is often called *ridge regression*. The regularisation term is typically chosen to be some norm on the function f, and in our case we choose it to be a norm on the space $\mathcal{F}$. This turns our minimization problem (1) into the problem of finding $\hat{f}$ such that:

$$\hat{f} = \underset{f\in\mathcal{F}}{\operatorname{argmin}} \left(\frac{1}{n}\sum_{i=1}^{n}(f(x_i) - y_i)^2 + \lambda\|f\|_{\mathcal{F}}^2\right) \tag{2}$$

where $\lambda > 0$ is a regularisation parameter. Intuitively, one can think of this approach as balancing the accuracy of $\hat{f}$ on the training data points with the 'simplicity' of $\hat{f}$.

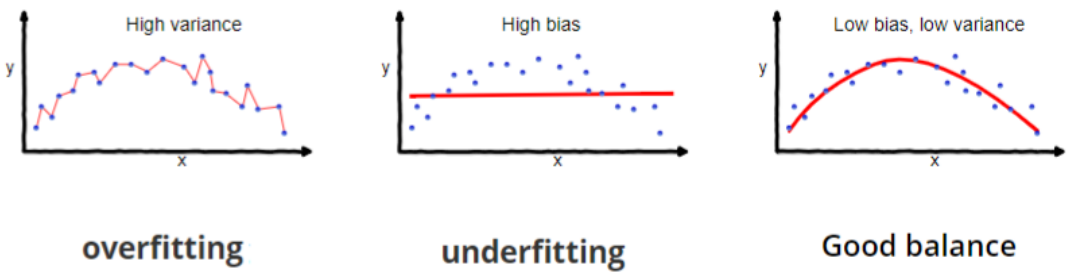


Figure 1: Visualization of the Bias-Variance Trade-off
Source: https://dapperdatadig.wordpress.com/2020/06/15/bias-and-variance/

Reproducing Kernel Hilbert Spaces

In order to define a norm on $\mathcal{F}$, however, we need to define some inner product on this space. This means that we are going to have to restrict the search space $\mathcal{F}$ to a subspace of all possible functions mapping from $\mathbb{R}^m \to \mathbb{R}$ and on this subspace we need to define an inner product. This restriction is a direct consequence of the fact that we want to have some notion of simplicity of $\hat{f}$. Still, at this point, we have no clear path to actually finding this function. To this end, we are going to put even more restrictions on our search space $\mathcal{F}$, by requiring that it is a so-called *Reproducing Kernel Hilbert Space*.

Definition 2.1. A function space $\mathcal{F}$ is a Reproducing Kernel Hilbert Space (RKHS) if:

- $\mathcal{F} = \overline{\mathcal{F}_0} := \overline{\text{span}}\{K(\cdot, x) \mid x \in \mathbb{R}^m\}$ for a symmetric, positive definite function $K : \mathbb{R}^m \times \mathbb{R}^m \to \mathbb{R}$ (called the *kernel function*)
  - Symmetry means that $K(x, y) = K(y, x)$ for all $x, y \in \mathcal{F}$
  - Positive definiteness means that $\sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j K(x_i, x_j) \geq 0 \; \forall x_i, x_j \in \mathbb{R}^m$ and $\forall \alpha_i, \alpha_j \in \mathbb{R}$
- $\mathcal{F}$ is a Hilbert space, i.e.,
  - $\mathcal{F}$ is an inner product space, which in this case is given by $\langle f, g \rangle_{\mathcal{F}} = \left\langle \sum_{i=1}^n \alpha_i K(\cdot, x_i), \sum_{j=1}^m \beta_j K(\cdot, x_j) \right\rangle_{\mathcal{F}} := \sum_{i=1}^n \sum_{j=1}^m \alpha_i \beta_j K(x_i, x_j)$
  - $\mathcal{F}$ is a complete space (i.e. the limit of any Cauchy sequence in $\mathcal{F}$ also lies in $\mathcal{F}$), which is guaranteed by the definition of $\mathcal{F}$ as the closure of $\mathcal{F}_0$

Changing the space $\mathcal{F}$ in (2) results in the *RKHS minimization problem*: find $\hat{f}$ such that

$$\hat{f} = \underset{f \in \mathcal{F}}{\text{argmin}} \left(\frac{1}{n} \sum_{i=1}^n (f(x_i) - y_i)^2 + \lambda \|f\|_{\mathcal{F}}^2 \right) \tag{3}$$

where $x_i \in \mathbb{R}^m$, $y_i \in \mathbb{R}$ for $i = 1, 2, ..., n$ and $\lambda \in \mathbb{R}_{>0}$ and $\mathcal{F}$ is a RKHS.

This may feel like a rather cumbersome restriction, but, as we will see shortly, this specific structure facilitates an approach where the solution can be found by solving a matrix equation.

Selecting $\mathcal{F}$ and K

One of the difficulties in restricting a general optimisation problem to a RKHS minimization problem, is the question of what kernel function K should be used for a given set of data points and/or search space $\mathcal{F}$. One interesting result in this consideration, is the following theorem:

Theorem 2.2. (Moore-Arenszajn) For any RKHS $\mathcal{F}$ of functions $f : \mathbb{R}^m \to \mathbb{R}$, there exists a unique symmetric positive definite kernel function $K : \mathbb{R}^m \times \mathbb{R}^m \to \mathbb{R}$ such that $\mathcal{F} = \overline{\text{span}}\{K(\cdot, x) \mid x \in \mathbb{R}^m\}$.

This means that, once we have decided on an appropriate space $\mathcal{F}$, there is only one corresponding kernel function K that we could choose (provided that $\mathcal{F}$ is in fact a RKHS).

In practice, however, the best choice for $\mathcal{F}$ is not necessarily known beforehand and finding the corresponding kernel function is not trivial. For this reason, it is common practice to not explicitly choose $\mathcal{F}$, but instead straight-up pick the kernel function. Using past experience and intuitions on the underlying function g, one is typically able to choose an appropriate kernel function (and implicitly an appropriate space $\mathcal{F}$). Some of the more popular choices (see [El Khatib and Alkhatib, 2024]) are the following:

- Linear kernel: $K(x_1, x_2) = x_1^\top x_2$
- Polynomial kernel: $K(x_1, x_2) = (1 + c x_1^\top x_2)^d$ with $c > 0$ and polynomial degree $d \in \mathbb{N}$
- Gaussian (RBF) kernel: $K(x_1, x_2) = e^{-c\|x_1 - x_2\|_2^2}$ with $c > 0$
- Laplacian kernel: $K(x_1, x_2) = e^{-c\|x_1 - x_2\|_1}$ with $c > 0$

In practice, the constants c are often used to tune the method and prevent under- or over-fitting. In our context, however, we are also free to vary λ (which serves the same purpose) and hence we decide to fix $c = 0.1$ throughout this report (based on the experimental results in [El Khatib and Alkhatib, 2024]).

Kernel Ridge Regression

Despite adding this structure to the problem, the space $\mathcal{F}$ in which we search for the function f might very well still be infinite dimensional (depending on the specific kernel function). This makes it very hard, if not impossible, to minimize over this space.

Luckily, the structure of the RKHS allows use to rewrite the problem in such a way that the space over which we minimize, becomes finite-dimensional! To this end, we introduce the Representer Theorem.

Theorem 2.3. (Representer Theorem) The minimizer $\hat{f}$ of the RKHS minimization problem must be of the form $\hat{f}(\cdot) = \sum_{i=1}^{n} \hat{\alpha}_i K(x_i, \cdot)$ for $\hat{\alpha} \in \mathbb{R}^n$, where K is the kernel function that defines $\mathcal{F}$.

Due to this theorem, the RKHS minimization problem (3) can be rewritten in terms of $\alpha \in \mathbb{R}^n$:

$$\begin{aligned}
&\text{finding the function } \hat{f} \in \mathcal{F} \text{ such that } \hat{f} = \operatorname*{argmin}_{f \in \mathcal{F}} \left(\frac{1}{n} \sum_{i=1}^{n} (f(x_i) - y_i)^2 + \lambda \|f\|_{\mathcal{F}}^2 \right) \\
&\iff \\
&\text{finding the vector } \hat{\alpha} \in \mathbb{R}^n \text{ such that } \hat{\alpha}^* = \operatorname*{argmin}_{\alpha \in \mathbb{R}^n} \left(\frac{1}{n} \|K\alpha - y\|_2^2 + \lambda \alpha^\top K \alpha \right)
\end{aligned} \tag{4}$$

Here (with a slight abuse of notation) K is an n by n matrix with $K(x_i, x_j)$ as its (i, j)-th entry. Note that due to the positive definiteness and symmetry of the kernel function K (Definition 2.1), this matrix K is also positive definite and symmetric.

To justify this equivalence (4), we work out the following two equalities.

Equality 1:

$$\begin{aligned}
\sum_{i=1}^{n} \left(\hat{f}(x_i) - y_i \right)^2 &\overset{\text{Thm 2.3}}{=} \sum_{i=1}^{n} \left(\sum_{j=1}^{n} \hat{\alpha}_j K(x_j, x_i) - y_i \right)^2 \overset{\text{Def 2.1}}{=} \sum_{i=1}^{n} \left(\sum_{j=1}^{n} \hat{\alpha}_j K(x_i, x_j) - y_i \right)^2 \\
&= \sum_{i=1}^{n} \left((K\hat{\alpha})_i - y_i \right)^2 = \|K\hat{\alpha} - y\|_2^2
\end{aligned}$$

Equality 2:

$$\begin{aligned}
\|\hat{f}\|_{\mathcal{F}}^2 &= \left\langle \sum_{i=1}^{n} \hat{\alpha}_i K(x_i, \cdot), \sum_{i=1}^{n} \hat{\alpha}_i K(x_i, \cdot) \right\rangle_{\mathcal{F}} \overset{\text{Def 2.1}}{=} \sum_{i=1}^{n} \sum_{j=1}^{n} \hat{\alpha}_i \hat{\alpha}_j K(x_i, x_j) \\
&= \sum_{i=1}^{n} \hat{\alpha}_i \sum_{j=1}^{n} \hat{\alpha}_j K(x_i, x_j) = \sum_{i=1}^{n} \hat{\alpha}_i (K\hat{\alpha})_i = \hat{\alpha}^\top K \hat{\alpha}
\end{aligned}$$

We have now turned the RKHS minimization problem (3) into the what is is called the *Kernel Ridge Regression* (KRR) minimization problem: find $\hat{\alpha}$ such that

$$\hat{\alpha} = \operatorname*{argmin}_{\alpha \in \mathbb{R}^n} \left(\frac{1}{n} \|K\alpha - y\|_2^2 + \lambda \alpha^\top K \alpha \right) \tag{5}$$

where $\lambda \in \mathbb{R}_{>0}$, $y \in \mathbb{R}^n$, and $K \in \mathbb{R}^{n \times n}$ defined by some kernel function K and n data points $x_i \in \mathbb{R}^m$.

Solution to the KRR problem

To solve this minimization problem, we observe that $\frac{1}{n}\|K\alpha - y\|_2^2 + \lambda \alpha^\top K \alpha \geq 0$ for all $\alpha \in \mathbb{R}^n$, for all $x_1, x_2, ..., x_n \in \mathbb{R}^m$ and for all $y \in \mathbb{R}^n$. This is because $\lambda, n > 0$ and all other terms are norms and/or squares. Because this is a quadratic, non-negative function of α, it is convex and thus $\hat{\alpha}$ is a minimizer if, and only if, $\frac{d}{d\alpha}(\frac{1}{n}\|K\hat{\alpha} - y\|_2^2 + \lambda(\hat{\alpha})^\top K \hat{\alpha}) = 0$. Writing this out, gives rise to the following:

$$\begin{aligned}
&\frac{d}{d\alpha} \left(\frac{1}{n} \|K\hat{\alpha} - y\|_2^2 + \lambda \hat{\alpha}^\top K \hat{\alpha} \right) = \frac{2}{n} K^\top (K\hat{\alpha} - y) + 2\lambda K \hat{\alpha} \overset{K \text{ sym.}}{=} \frac{2}{n} K (K\hat{\alpha} - y) + 2\lambda K \hat{\alpha} \\
&= 2K \left(\frac{1}{n} (K\hat{\alpha} - y) + \lambda \hat{\alpha} \right) = 0 \\
&\iff 2K \Big((K\hat{\alpha} - y) + n\lambda \hat{\alpha} \Big) = 0 * n = 0 \\
&\iff K\hat{\alpha} - y + n\lambda \hat{\alpha} = 0 \qquad \text{(Since } K \text{ is positive definite, which implies } \ker(K) = \{0\}) \\
&\iff (K + n\lambda I)\hat{\alpha} = y
\end{aligned}$$

So, the solution $\hat{\alpha}$ to the KRR minimization problem (5) can be found by solving the linear system

$$(K + n\lambda I)\hat{\alpha} = y \tag{6}$$

3 Classical Approach

As described in Section 2, the KRR minimization problem boils down to solving the linear system (6): $(K + n\lambda I)\hat{\alpha} = y$. The matrix K is constructed from a kernel function, which means that K is symmetric and positive definite. By adding the diagonal matrix $n\lambda I$ to K, we clearly maintain the symmetry, and also the positive definiteness since:

$$x^\top Kx > 0 \quad \forall x \neq 0 \quad \Longrightarrow \quad x^\top (K + n\lambda I)x = x^\top Kx + n\lambda x^\top x > n\lambda x^\top x > 0 \quad \forall x \neq 0$$

The general approach to solving such a system, is to construct a Cholesky decomposition $A = LL^\top$ (with A a symmetric positive definite, and L lower triangular) and efficiently solving the resulting system use triangular solving. This approach, however, is relatively susceptible to numerical instabilities. If the singular values of the system matrix are very close to zero, the numerical process of the Cholesky decomposition combined with rounding errors, could alter the matrix such that one or more of the singular values become almost basically equal to zero, or even negative., This would cause the matrix to become singular or indefinite, respectively., which breaks the presumptions of using the Cholesky decomposition to solve the linear system.

Given the fact that we encounter very small singular values in the setting of KRR minimization (see section 4), we therefore switch to the so-called Bunch-Kaufman approach. This algorithm is considered to be highly numerically stable (see [Higham, 1997]) while still having a computational cost comparable to that of the highly efficient Cholesky decomposition (see [Bunch and Kaufman, 1977]).

In this section, we introduce the Bunch-Kaufman approach for solving a symmetric linear system and use it to solve the KRR minimization problem. We then investigate the performances of different kernel functions on actual regression problems, and investigate the computational cost of each part of this classical approach.

3.1 Theory: Bunch-Kaufman approach to solving a symmetric linear system

The go-to, classical direct approach for solving a linear system of the form $Ax = b$ with A being a symmetric (possibly indefinite) matrix, is the so-called Bunch-Kaufman decomposition. This approach is a variation on the $LDL^\top$ decomposition (as described in [Kaw et al., 2009]) for symmetric, positive definite linear systems. For this reason, we start by introducing and analysing the $LDL^\top$ approach.

The LDL^T approach

Any symmetric, positive definite matrix $A \in \mathbb{R}^{n\times n}$ can be decomposed as $A = LDL^\top$, where L is a lower triangular matrix with ones on the diagonal, and D is a diagonal matrix. In this case, solving the linear system $Ax = b$ can then be performed efficiently by using the triangular and diagonal structures of the decomposition. Visually, the $LDL^\top$ decomposition looks as follows:

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix} = \begin{bmatrix} 1 & 0 & \dots & 0 \\ l_{21} & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ l_{n1} & l_{n2} & \dots & 1 \end{bmatrix} \begin{bmatrix} d_{11} & 0 & \dots & 0 \\ 0 & d_{22} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & d_{nn} \end{bmatrix} \begin{bmatrix} 1 & l_{21} & \dots & l_{n1} \\ 0 & 1 & \dots & l_{n2} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix}$$

By multiplying these right three matrices, we obtain the following formulas for construction the matrices D and L:

$$d_{jj} = a_{jj} - \sum_{k=1}^{j-1} l_{jk}^2 d_{kk} \qquad l_{ij} = \frac{1}{d_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik} d_{kk} l_{jk} \right) \tag{7}$$

The resulting system, $LDL^\top x = b$, can now be solved with the following procedure:

1) Find z such that $Lz = b$. $\begin{bmatrix} 1 & 0 & \dots & 0 \\ l_{21} & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ l_{n1} & l_{n2} & \dots & 1 \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix} \quad z_i = b_i - \sum_{k=1}^{i-1} l_{ik} z_k \quad i = 1, ..., n$

$$
\begin{aligned}
&\text{2) Find } y \text{ such that } Dy = z: \quad \begin{bmatrix} d_{11} & 0 & \dots & 0 \\ 0 & d_{22} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & d_{nn} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_n \end{bmatrix} \quad y_i = \frac{z_i}{d_{ii}} \quad i = 1, ..., n \\
&\text{3) Find } x \text{ such that } L^\top x = y: \quad \begin{bmatrix} 1 & l_{21} & \dots & l_{n1} \\ 0 & 1 & \dots & l_{n2} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} \quad x_i = y_i - \sum_{k=i+1}^{n} l_{ki} x_k \quad i = n, ..., 1
\end{aligned}
\tag{8}
$$

Indeed, $Ax = LBL^\top x = LDy = Lz = b$.

To determine how the computation cost of this approach scales with n, we can compute the number of required flops to construct the $LDL^\top$ decomposition and perform the system solving steps. The full computations can be found in the appendix, but the conclusion is that this approach requires $\sim \frac{1}{3}n^3$ flops. Note that the standard direct approach for non-symmetric matrices (pivoted LU decomposition) requires $\sim \frac{2}{3}n^3$ flops, which is twice as much. By making use of the symmetry, we have thus essentially halved the computational cost, which seems to make a lot of sense intuitively.

The Bunch-Kaufman approach

In order to solve a symmetric linear system that is not necessarily positive definite (i.e. possibly indefinite), the $LDL^\top$ approach has to be altered such that positive definiteness is no longer required. The most commonly used approach is described in [Bunch and Kaufman, 1977] and is generally referred to as the Bunch-Kaufman decomposition.

We will not introduce the precise details here, but the main idea of this approach is to use 1×1 and 2×2 diagonal pivots. This results in a permutation matrix P such that $PAP^\top = L\tilde{D}L^\top$, where $\tilde{D}$ is a block diagonal matrix with blocks of size 1 or 2. With this decomposition, the linear system $Ax = b$ can still be solved very efficiently. The use of 2×2 pivots instead of just 1×1 pivots, is needed to prevent the size of the entries of the system matrix from growing too fast during the pivoting process, which would cause instability.

Compared to the regular $LDL^\top$ approach, the Bunch-Kaufman approach clearly requires additional flops to solve the symmetric linear system, but not all that much more. In fact, the computational cost remains $\sim \frac{1}{3}n^3$, which is the same cost as that of the Cholesky decomposition approach. This means that the Bunch-Kaufman algorithm is indeed comparatively computationally expensive.

3.2 Experiment: Performance of Kernels on Regression Problems

Having introduced the KRR minimization problem and the classical way of solving it (using Bunch-Kaufman), we are going to test this approach on some actual regression problems. The goal of this experiment is to test the performance of the KRR minimization (using the different kernel functions mentioned in section 2) and investigate the effect of the number of training data points (n) and the size of the regularisation parameter (λ) on the accuracy of the resulting predictions.

3.2.1 Experimental set-up

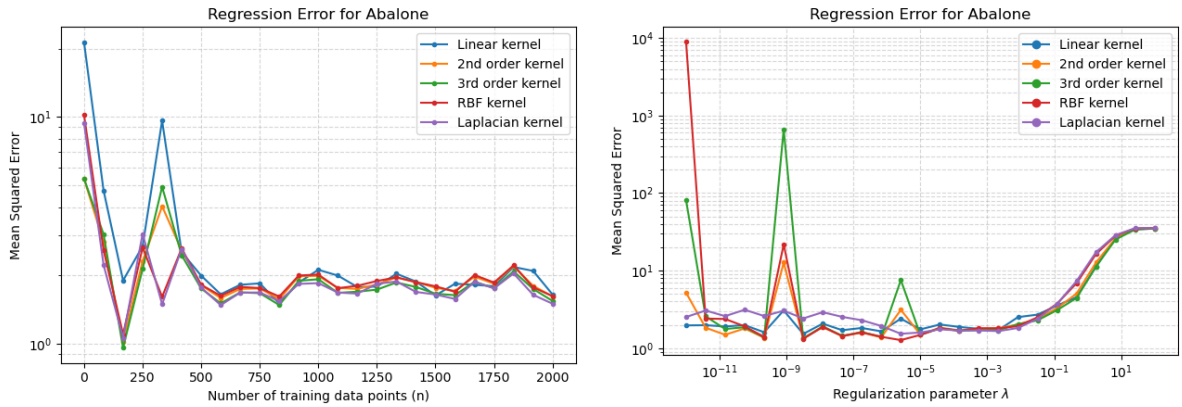
The considered kernels are all of the kernel functions described in section 2. The polynomial kernel function is considered for degrees $d = 2$ and $d = 3$. Three different regression data sets are used: Abalone, Pumadyn-8fm, and Pumadyn-8nh. A detailed description of these data sets and the expected behaviour of their underlying function g can be found in the appendix.

After the training phase with n data points, the resulting predictor will be presented with an additional $n_{test}(= 0.3n)$ testing data points. To asses the performance, the predictions y_{pred} will be compared with the actual values y_{test}, leading to the mean squared error: $\frac{1}{n_{test}} \sum_{i=1}^{n_{test}} (y^i_{test} - y^i_{pred})^2$.

At first, the value of λ is fixed at 10^{-3} (based on the experiments in [Alaoui and Mahoney, 2015]) and the value of n is varied. Afterwards, the value of n is fixed at 2000 (based again on [Alaoui and Mahoney, 2015])

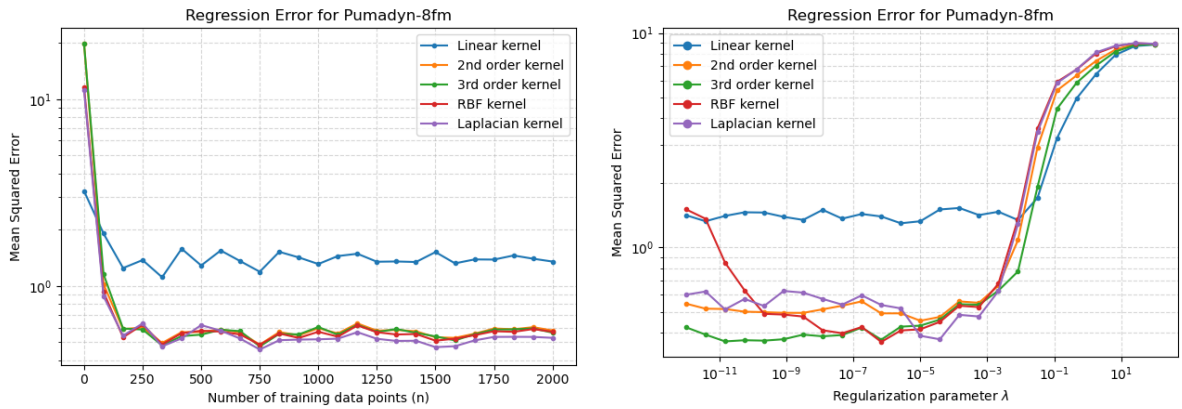
and the value of λ is varied. To improve the reliability of the result, we repeat each combination of kernel function, n and λ a total of 3 times, and consider the resulting average of the mean squared errors.

3.2.2 Results



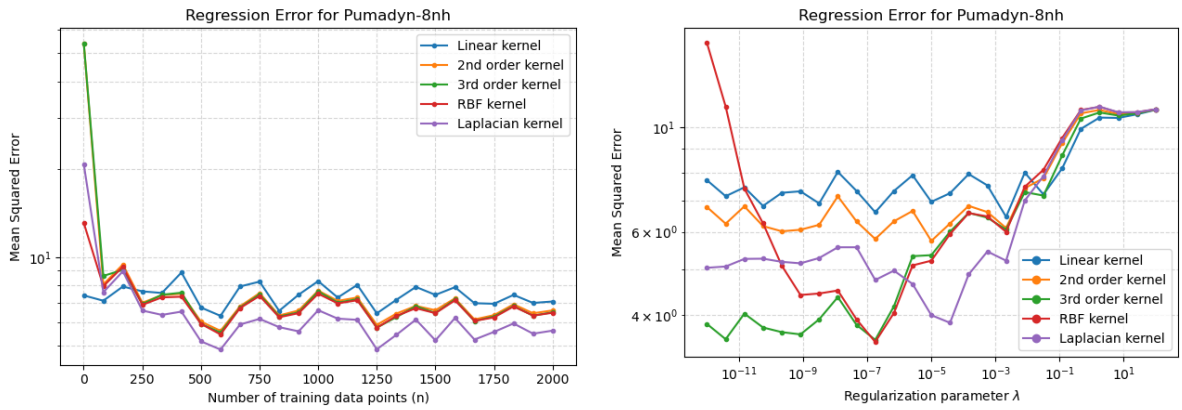
(a) Effect of n on error ($\lambda = 10^{-3}$, $n_{test} = 0.3n$) (b) Effect of λ on error ($n = 2000$, $n_{test} = 0.3n$)

Figure 2: Experiment results for the Abalone data set



(a) Effect of n on error ($\lambda = 10^{-3}$, $n_{test} = 0.3n$) (b) Effect of λ on error ($n = 2000$, $n_{test} = 0.3n$)

Figure 3: Experiment results for the Pumadyn-8fm data set



(a) Effect of n on error ($\lambda = 10^{-3}$, $n_{test} = 0.3n$) (b) Effect of λ on error ($n = 2000$, $n_{test} = 0.3n$)

Figure 4: Experiment results for the Pumadyn-8nh data set

3.2.3 Discussion

Looking at the plots of the results (2, 3 and 4), we observe that the orders of magnitude of the mean squared errors are very reasonable given the range of the target values for each of the data sets. We therefore conclude that KRR minimization is a good approach to solving these regression problems.

A rather surprising result, is the relatively poor performance of the linear kernel function on the Pumadyn-8fm data set (see figure 3). The underlying function g is supposed to be somewhat linear and hence we would have expected this kernel to perform better. A possible reasons for this behaviour, is the presence of noise, which the other kernel functions might be better suited for.

Effect of n on the performance of the predictor

Looking at plots 2a, 3a and 4a, we observe a general trend in the effect that n has on the mean squared error: lower values of n result in higher errors, while higher values of n result in lower errors. Intuitively, this makes sense because having more data points available for training, should allow for more accurate predictions.

There does, however, also appear to be some sort of plateau in how much increasing n can improve the accuracy of the predictors. The existence of such a plateau is not surprising, as there is a limit on how well a specific kernel function can predict the underlying function g. Both because of a mismatch in the type of function (a linear kernel cannot fully describe a quadratic g) and because of the presence of noise. Simply using more training data points cannot not fully overcome these issues. The precise value of n at which this plateau is reached seems to depend on the specific data set and might also still depend on the specific value of λ that is used. To be on the safe side while trying to find the best possible predictor, it thus makes sense to set n as large as possible in practice. This will of course affect the computational cost, which we will come back to in the next experiment.

Effect of λ on the performance of the predictor

Looking at plots 2b, 3b and 4b, we observe a general trend in the effect that λ has on the mean squared error: large values of λ result in higher errors, while very small values of λ may also result higher errors. The latter is most visible for the RBF kernel, but not so much for the linear and polynomial kernels. This general behaviour makes sense, because higher values of λ give more importance to the 'smallness' of the predictor (increasing the risk of under-fitting), while lower values of λ give more importance to precisely predicting the training data (increasing the risk of over-fitting).

Based on this experiment, the optimal λ generally seems to be somewhere between 10^{-7} and 10^{-3}, depending on the data set and the specific kernel function. Also, some kernel functions (mainly the RBF, but also the Laplacian) appear to be more sensitive to the tuning of λ and in particular more susceptible to over-fitting. On the other hand, these two kernels do show very good performance in terms of the mean squared error on all three data sets. In practice there might thus some trade-off to be made between kernel functions with the overall best peak performance (probably RBF or Laplacian) and those least sensitive to the tuning of the regularisation parameter λ (probably the linear and polynomial kernels).

A general approach to preventing over-fitting or under-fitting, is to use N-fold cross-validation. By repeatedly performing the KRR minimization for increasingly smaller values of λ, we can try to find the optimal λ. In order for this to work, we do however need to test our resulting approximation $\hat{f}$, but using the testing set for this purpose is a no-go (this may really only be after the training phase has been completed). Instead we split our training data into N parts, and at each repeated KRR minimization, we keep out one of the parts for cross-validation. On this testing fold, we compute the mean squared error and as we keep decreasing the value of λ, this error should go down, until it might at some point increase. At this point, we would be starting to over-fit our model and have reached the bottom of the λ-valley, which is where we want to be. For a more detailed description, see https://en.wikipedia.org/wiki/Cross-validation_(statistics), but the main take-away is that, through this approach, we can tune λ during the training process such that it is near-optimal, which leads to the lowest achievable prediction errors.

3.3 Experiment: Computational Cost of Classical Approach

The goal of this experiment is to investigate the computational cost of each of the parts of the classical approach to solving the KRR minimization problem. These parts are: (1) constructing the system matrix $K + n\lambda I$ and (2) solving the linear system $(K + n\lambda I)\hat{\alpha} = y$.

The computational performance should not depend on the chosen λ, the kernel function parameters c, or the precise data set, as those only affect the accuracy of the predictions. Instead, we are interested in the effect of the kernel function used and the influences of the number of training data points (n) and the number of data features (m).

Performing the computational cost analyses in terms of flops (see appendix), we can find expected costs. Based on these computations, we expect to see the following:

a) The cost of solving the linear system scales like n^3.
b) The cost of constructing K scales like $c_i n^2$ for a fixed m.
c) The cost of constructing K scales like $c_i m$ for a fixed n.
d) The constants c_i are such that $c_1 < c_2 < c_3 \quad << \quad c_L < c_R$, where the c_i's correspond to the linear, second order, third order, Laplacian and RBF kernel respectively. This expected gap between the linear/polynomial and RBF/Laplacian kernels stems from the cost of computing an exponential.

3.3.1 Experimental set-up

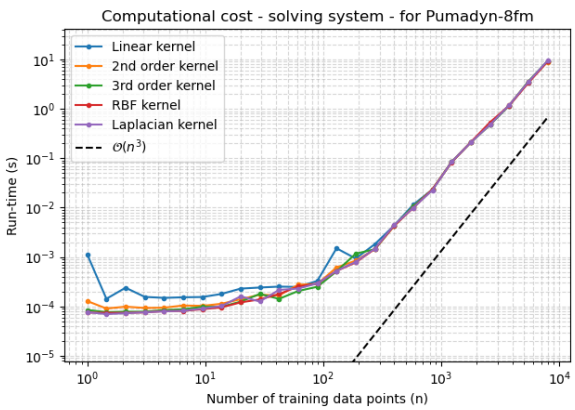
The regularisation parameter is fixed as $\lambda = 10^{-3}$ and the considered kernel functions are precisely the same as those described in section 3.2.1. The only effect that the choice of data set has on the experiment, is the number of features (m) of each data point. Since we can vary m by duplicating features, only a single data set is needed for this experiment, which we pick to be the Pumadyn-8fm data set (see appendix).

The computational cost of each of the two parts of the algorithm is assessed in terms of run-time in seconds. For this purpose the python package *time* is used. The timing is only performed on these two parts, and thus does not include the time needed to set up the experiment, sample the training data points, the preprocessing of this data, or any of the plotting.

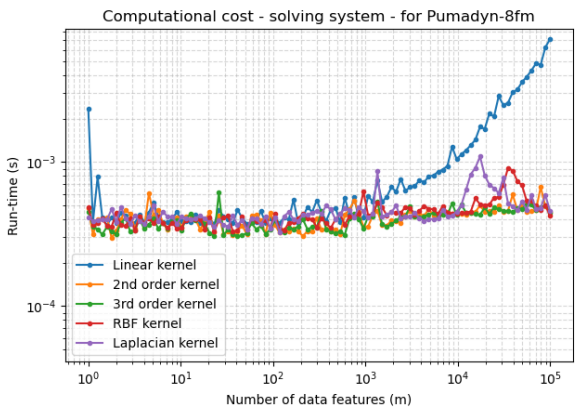
Firstly, the algorithm is run for increasingly larger values of n (number of training data points) for each of the five kernel functions and the run-time of the two parts is measured. For now, we fix the number of data features at $m = 8$, which is done by simply running this experiment of the Pumadyn-8fm data set.

Secondly, the algorithm is run for increasingly larger values of m (number of data features) for each of the five kernel functions and the run-time of the construction of the system matrix is measured. The number of data points is fixed at $n = 100$. To construct data points with the desired number of features, we either use the first m columns of the data set (if $m < 8$) or we duplicate as many of the columns as we need (if $m > 8$).

3.3.2 Results

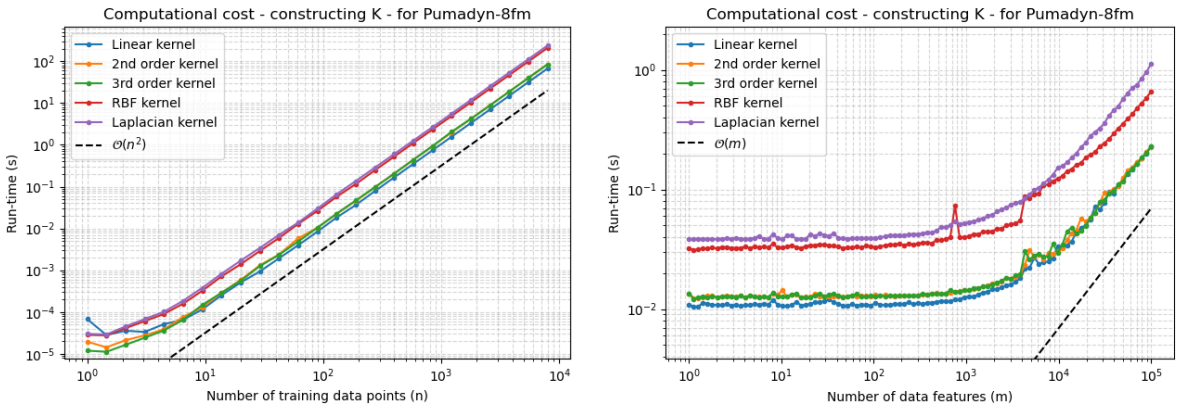


(a) Effect of n on the computational cost ($m = 8$)



(b) Effect of n on the computational cost ($m = 8$)

Figure 5: Experiment results for solving the linear system



(a) Effect of n on the computational cost ($m = 8$) (b) Effect of m on the computational cost ($n = 100$)

Figure 6: Experiment results for constructing the system matrix K

3.3.3 Discussion

Effect of the kernel functions on solving the linear system

When looking both at plot 5a and plot 5b, we observe that all the kernel functions result in a very similar computational cost for solving the linear system, except for the linear kernel. Apart from the behaviour of the linear kernel, this is not surprising, because the kernel functions only affect the entries of the matrix K and the run-time of the linear solver should not depend on the specific entries of the system matrix. Any fluctuations or differences between the kernel functions should thus be the result of noise in the computation process and the measuring of the run-time.

For the lower values of n and higher values of m, however, we observe that the linear kernel function consistently results in significantly longer run-times for the linear solver. Due to the consistency and the substantial difference with the other kernel functions, this cannot fully be attributed to noise. Our best explanation for this observation, is that it is the result of some optimisation that is performed during the compiling of the code.

In the Python script, all initializations are done before we loop over the kernel functions and perform the measurements. It could however very well be the case that the compiler decided to perform some tasks only during the first iteration of the loop (such as loading the training data points into memory), which results in lower run-times for the remaining iterations. This would be a plausible explanation, given the fact that the linear kernel is used precisely in the first iteration of the loop, and that the computational cost for solving the linear system for the linear kernel scales with m in plot 5b. The reason this additional cost does not seem to scale with n in 5a, would be that this additional run-time scales by a lower order than the run-time for solving the linear system.

To verify this explanation, we have re-run this experiment with the second order kernel as the first iteration of the loop. This time around, we observed the same behaviour as before, but for the second order kernel instead of the linear kernel. Hence we conclude that our explanation for this phenomenon is most likely correct, and during the remaining of this discussion, we consider the additional computational cost for the linear kernel to be non-existent.

Effect of n on solving the linear system

Looking at plot 5a, we observe that, for sufficiently high values of n, the computational cost of solving the linear system scales like n^3. This corresponds precisely to the expected behaviour. The fact that this is not visible for the lower values of n, is likely due to the lower order terms (such as n^2 and n) still having a significant impact. As n increases, the n^3 term becomes increasingly more dominant and hence it becomes more visible in the results.

Effect of m on solving the linear system

Looking at plot 5b, we observe that the computational cost of solving the linear system does not scale

with m. This corresponds precisely to the expected behaviour, namely that the number of required flops for solving the linear system does not depend on the number of data features m.

Effect of the kernel functions on constructing the system matrix K

When looking both at plot 6a and plot 6b, we observe that all kernel functions show similar behaviour. When inspecting the lower values of n, it seems likely that the aforementioned compiler-related distortion is again present in these results.

The linear, second order and third order kernel show always produce a very similar computational cost, as do the RBF and Laplacian kernel. Also, it indeed seems to be the case that $c_1 < c_2 < c_3$, which is especially visible in 6b where (for all values of m) the computational cost of the linear kernel is lower than that of the second and third order kernel. The difference between the latter two is however not very large. There also is a significant different between the run-times for linear/polynomial kernels and that of the RBF/Laplacian kernels. All this corresponds precisely to the expected behaviour.

The only observation that does not match the expected behaviour, is the fact that the RBF kernel always (for any n and any m) yields a lower computational cost than the Laplacian. Based on the computational analysis in terms of flops, we expected this to be the other way around. The most likely explanation for this, is that the *numpy* implementation of the Euclidean norm is in fact computationally cheaper than that of the L1-norm.

Effect of n on constructing the system matrix K

Looking at plot 6a, we observe that the computational cost of constructing K scales like n^2. This corresponds precisely to the expected behaviour. The fact that this is already visible for small values of n, is likely due to the lower order terms (such as n) being so small, that they are already being dominated by the n^2 term.

Effect of m on constructing the system matrix K

Looking at plot 6b, we observe that the computational cost of solving the linear system scales like m (i.e. linearly). This corresponds precisely to the expected behaviour. Again, the fact that this is not visible for the lower values of m, is likely due to the lower order terms (i.e. the constants) still having a significant impact. As m increases, the m term becomes increasingly more dominant and hence it becomes more visible in the results.

Overall computational cost

When looking at the overall computational cost of solving the KRR minimization problem, we observe that computing the kernel matrix K takes up a large amount of time. As a particular example, consider the run times for the fixed $m = 8$ in the plots 5a and 6a. We see that for all five kernels, the run-time needed to compute K is higher than the run-time needed to solve the linear system, at least for $n \leq 10^4$.

This relative cost of constructing the kernel matrix only gets worse as more features are considered. The cost of solving the linear system stays constant (it does not depend on m), while the cost of constructing K increases linearly.

This was not initially obvious, as the cost of constructing K scales like n^2, while the cost of solving the system scales like n^3. Eventually, the cost of solving the system will overtake the cost of constructing K, but even then the cost of constructing K is still considerable.

Thus, it can generally be assumed that the cost of constructing the system matrix K is a major computational bottleneck in solving the KRR minimization problem. Only for really large numbers of training data points (n) relative to the number of data features (m), does the cost of computing K become negligible or small compared to the cost of solving the linear system.

4 Low-rank approximations of the kernel matrix

As is often stated in literature (see e.g. [Alaoui and Mahoney, 2015]), the kernel matrix K typically has a so-called low-rank property. This means that the singular value distribution of the matrix exhibits a large drop. In other words: there is a (somewhat) clear separation point between a cluster of relatively large singular values and a cluster of relatively small singular values. This property opens the door to using low-rank approximations to K in order to solve the KRR problem at a much lower computational cost.

In this section, we set out to explore this low-rank property of kernel matrices and test the regression performance when using a low-rank approximation. Additionally, we explore how this low-rank property depends on the specific kernel function, on the underlying data set, on the number of training data points (n), and on the number of data features (m).

4.1 Experiment: singular value distribution of kernel matrices

The goal of this experiment is to analyse the distribution of the singular values of kernel matrices for different kernel functions on different data sets.

4.1.1 Experimental set-up

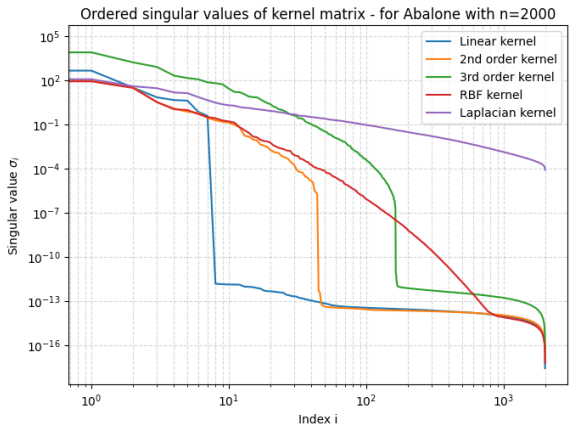
The considered kernel functions and data sets, are precisely the same as those described in section 3.2.1. For now, we decide to use all the features of these data sets (so $m = 8$) and a fixed amount of training data points $n = 2000$. No accuracy testing or system solving is performed, so $n_{test} = 0$ and the regularisation parameter λ is not used.

Besides these three data sets, we also consider the randomly generated data set as described in the appendix. This allows use to determine to what extend the low-rank-ness is a property of a kernel function and to what extend it depends on the considered data set. For these randomly generated data sets, we also pick $n = 2000$ and $m = 8$.

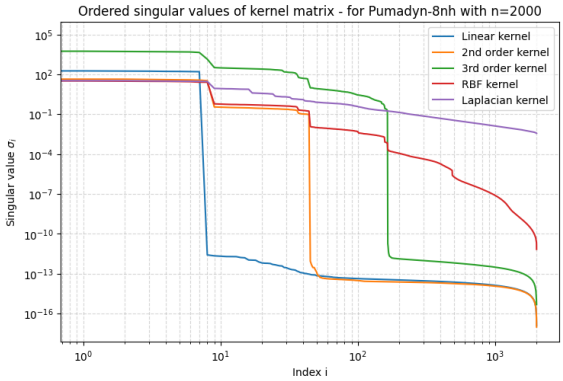
For each data set and kernel function, we compute the kernel matrix K using the $n = 2000$ training data points. We then compute the singular value decomposition (SVD) of K, such that $K = U\Sigma V^\top$, where Σ is a diagonal matrix containing the singular values of K. These singular values are sorted in descending order and this distribution is plotted in a graph with the sorted index on the x-axis and the size of the singular value on the y-axis.

Since the results from the Pumadyn-8fm and Pumadyn-8nh were partically identical, only the results for the Pumady-8nh data set is shown in the results section.

4.1.2 Results

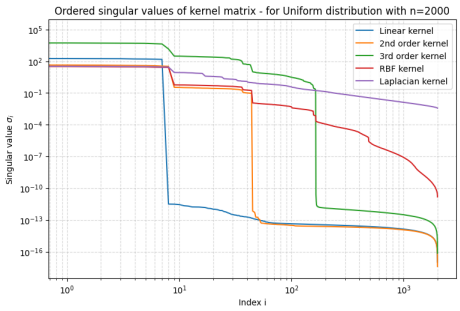


(a) Singular value distribution on "Abalone" dataset

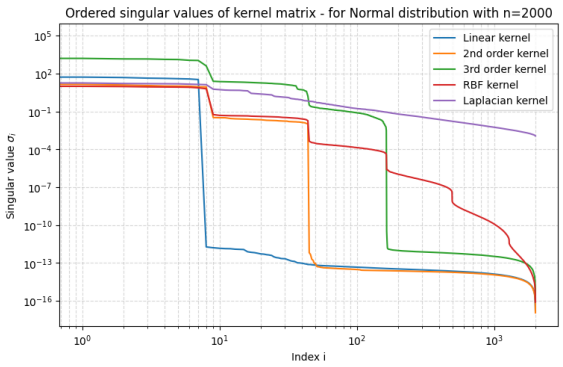


(b) Singular value distribution on "Pumadyn-8nh" dataset

Figure 7: Singular value distribution of K for different datasets and kernels



(a) Singular value distribution for uniformly distributed data points



(b) Singular value distribution for normally distributed data points

Figure 8: Singular value distribution of K for randomly distributed data points

4.1.3 Discussion

The distribution of singular values for the "Abalone" and "Pumadyn" datasets can be seen in Figure 7. First of all, notice that we see a lot of variety in the resulting distribution when considering different kernels. Interestingly, when considering the different datasets - with different underlying distribution of the data points - the differences are much more subtle.

For the polynomial kernels, we see clear drop-offs in the sizes of the singular values for both databases. For the RBF kernel we see a steady decrease, with in the case of "Pumadyn" with multiple relatively smaller drop-offs. For the Laplacian, there is a steady decrease where the smallest singular values are 'only' a factor of 10^4 smaller than the largest ones.

The differences between the different datasets are most visible for the RBF kernel. For the RBF kernel, the decrease of the sizes of the singular values occurs much more gradually in the "Abalone" dataset. This can also be seen to a lesser extent with the other four kernels. But the sudden large drop-offs are still there for these kernels, although these drop-offs are somewhat less pronounced. Another difference between the datasets is that the largest singular values of the "Abalone" dataset are a small factor higher than those of the "Pumadyn" dataset.

Another similarity between the distributions of the different datasets is that the drop-offs occur at the same points. For linear, these appear after the 8th index, for second order around the 45th index and for

the third order around 180th index. Also notice that for the "Pumadyn" dataset, there seems to be a correlation between drop-offs of different kernels, when looking at both the larger and smaller drop-offs.

So even though the underlying distributions of these datasets can be assumed to be very different, when considering the same kernel (and number of features), the singular value distributions show interesting similarities. This seems to indicate that the underlying distribution of data points has relatively little effect on the rank of the kernel matrix.

To see whether this is indeed the case we consider the results of Figure 8, with the singular value distributions of data points that correspond to an assigned distribution.

We have a distribution for the uniformly distributed points, and the normally distributed points. These underlying distributions are very different from each other, one has a constant density and one where the density has much variety. Nevertheless, we still see that the singular value distributions show minor differences. Still, the fact that the drop-offs, both large and small, occur at the same points is surprising. We see that these similarities can also be seen in comparison to "Abalone" and - most strongly - "Pumadyn".

To investigate the properties of the rank further, we will next study the accuracy of low-rank approximations of K.

4.2 Experiment: regression accuracy using low rank approximations

The goal of this experiment is to see how well a low-rank approximation of the kernel matrix can perform the regression task on a data set.

4.2.1 Experimental set-up

The considered kernel functions and data sets, are precisely the same as those described in section 3.2.1. We use all the features of these data sets (so $m = 8$) and a fixed amount of training data points $n = 2000$. The regularisation parameter is fixed as $\lambda = 10^{-3}$. The different kernel functions will have varying performances in terms of regression accuracy (as seen in section 3.2), but intuitively a sub-optimal λ should not affect how well each low-rank approximation approximates the kernel matrix K. For the kernel functions that require a parameter c, we have chosen the value $c = 0.1$. Except for the third order polynomial function, where (by accident) we have chosen $c = 1$.

To assess the accuracy on these regression problems, we take $n_{test} = 0.3 * n = 600$ testing data points. The performance of the kernels is again (see also section 3.2.1) assessed using the means squared error.

First, we compute the reference performances for each kernel, by just using the matrix K to solve the system and then computing the corresponding mean squared errors. The results are the coloured, dotted lines in the plots and represent the best level of performance we can expect the see from the low-rank approximations.

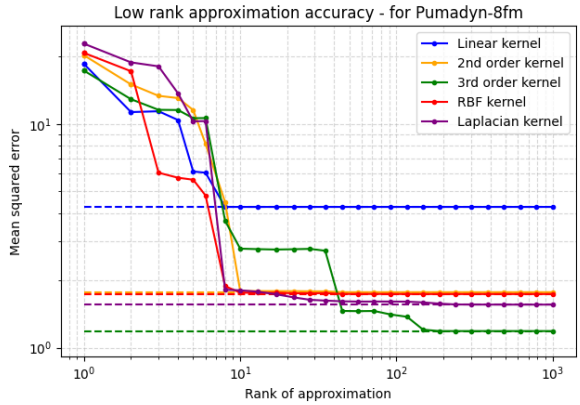
Secondly, we compute varying rank-k approximations to K such that $k < n$. A single rank k approximation is computed using the aforementioned singular value decomposition of A. If the singular values are on the diagonal are sorted from highest (in the top left) to lowest (in the bottom right) and the columns and rows of U (and $U^\top$) are sorted accordingly, the rank-k approximation to K is given by $\tilde{K} = U\Sigma_k(U^\top)$, where the first k columns of Σ_k contains the top left k by k block of Σ, and the rest is set to 0. The linear system $(\tilde{K} + n\lambda I)\hat{\alpha} = y$ is then solved as follows:

$$\begin{aligned}
(\tilde{K} + n\lambda I)\hat{\alpha} &= y \\
(U\Sigma_k U^\top + n\lambda I)\hat{\alpha} &= y \\
(\Sigma_k U^\top + n\lambda U^\top)\hat{\alpha} &= U^\top y \\
(\Sigma_k + n\lambda I)U^\top\hat{\alpha} &= U^\top y \\
U^\top\hat{\alpha} &= (\Sigma_k + n\lambda I)^{-1}U^\top y \\
\hat{\alpha} &= U(\Sigma_k + n\lambda I)^{-1}U^\top y,
\end{aligned}$$

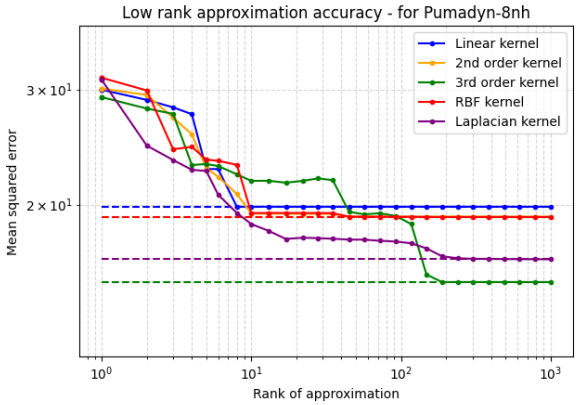
where we note that the inverse of $(\Sigma_k + n\lambda I)$ is easy to compute, as this is a diagonal matrix.

Subsequently, the mean squared error on the testing data points is computed. This all is done for a large range of values for the rank k, which is plotted against the resulting mean squared error.

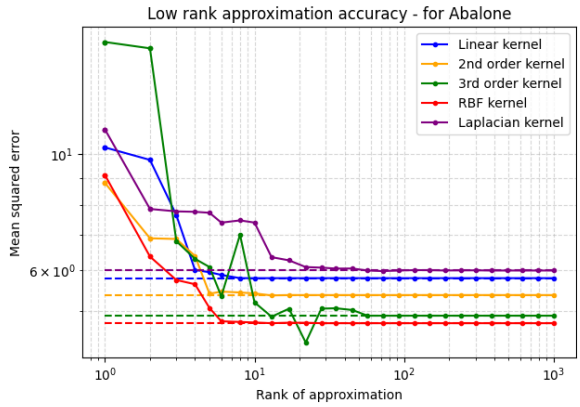
4.2.2 Results



(a) Accuracy of low-rank approximations of K for "Pumadyn-8fm"



(b) Accuracy of low-rank approximations of K for "Pumadyn-8nh"



(c) Accuracy of low-rank approximations of K for "Abalone"

Figure 9: Accuracy of low-rank approximations of K for different datasets and kernels

4.2.3 Discussion

For the linear kernel we see for every dataset that there is no loss in accuracy when we have an approximation of rank 8. For smaller rank approximations than this it differs per dataset how big the loss in accuracy is, but in each case the loss is visible in Figure 9. This is indeed what we would expect from the singular value distributions seen in Figure 7; as for each dataset, the drop between the 8th and 9th largest singular values is larger than 10^{10}.

For the second order polynomial kernel we see that a rank 10 approximation is already enough to at least be very close to the accuracy of K. When the approximation is of rank 45, we don't see a loss in accuracy for any dataset. This also coincides with the singular value distributions of Figure 7. It also shows that a relatively small drop of order between 10^2 and 10^3 might already be sufficient.

For the third order polynomial kernel, there is no loss visible for a 200 rank approximation in the case of the "Pumadyn" datasets. For the "Abalone" dataset this already occurs at a rank 60 approximation. Again we conclude that the relatively small differences in the singular value distributions can have quite a large effect on the required rank for a good approximation of K.

For the RBF kernel we see that a 10 rank approximation is very close to K. For the "Pumadyn-8nh" there is no loss in accuracy once the rank is of order 45, for the other datasets this point already occurs

sooner.

Finally, for the Laplacian kernel, a rank of 200 leads to no loss in accuracy. For "Abalone" this already occurs earlier. When comparing "Pumadyn-8fm" and "Pumadyn-8nh", it becomes clear that there are some differences despite the similarity in the singular value distribution. Between the rank of 10 and 200, the accuracy does not change much for both approximations. Nevertheless, the former is much closer to the accuracy of K compared to the latter.

In conclusion, for each kernel and dataset we see that the rank of the kernel matrix can be decreased considerably without any significant loss in its associated accuracy. For any dataset choosing the rank of the approximation as suggested by the kernel's singular value distribution seems to be a good choice. As this appears to guarantee a good approximation of K. In some cases a lower rank may be possible, although this differs per dataset. Note that while we have seen very similar singular value distributions for different datasets, we have not tested the effect that n and the number of features m has on this distribution.

4.3 Experiment: effect of n and m on singular value distribution

The goal of this experiment is to investigate the effect of the number of training data points n, and the number of features m on the singular value distribution of the associated kernel matrices.

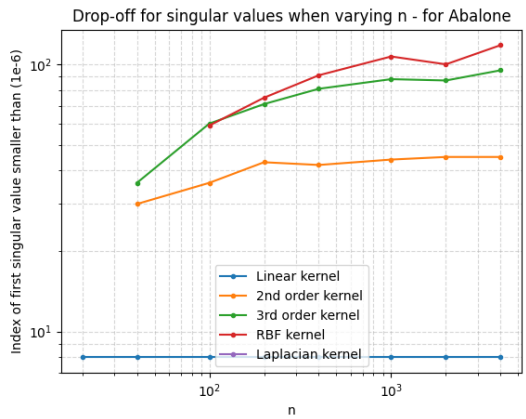
4.3.1 Experimental set-up

For the datasets we are using both the "Abalone" and the "Pumadyn-8nh" dataset. For each dataset we are considering all five kernels, where we choose as a constant $c = 0.1$ if needed.

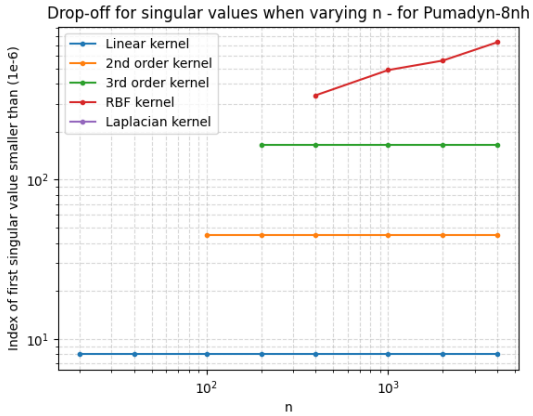
For each dataset we conduct two investigations. For the first, we fix the number of features m to be $m = 8$. We than vary n between $n = 20$ to $n = 4000$. For n that we consider we find the first index of the singular value that is smaller than 10^{-6}. We plot this index to the data points n. If for some n there are no singular values smaller than this number, then this point is not plotted.

For the second investigation, we fix the number of data points n to be $n = 2000$. We vary the number of features m between $m = 2$ and $m = 8$. As before, for every m we plot the index of the first singular that drops below 10^{-6}.

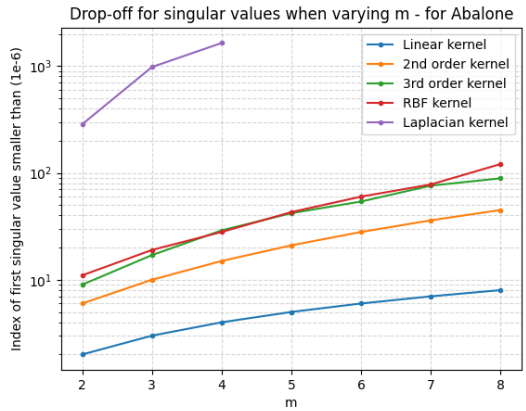
4.3.2 Results



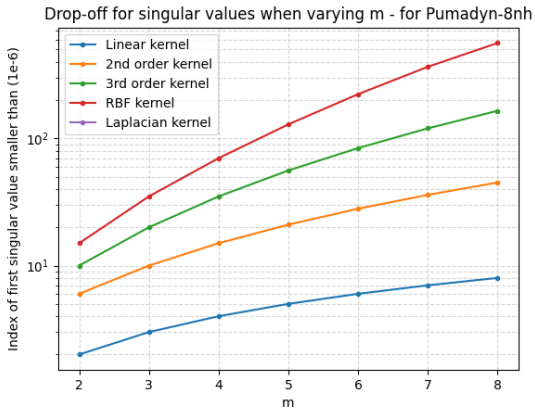
(a) Occurrence of drop-off when varying n on "Abalone"



(b) Occurrence of drop-off when varying n on "Pumadyn-8nh"



(c) Occurrence of drop-off when varying m (features) on "Abalone"



(d) Occurrence of drop-off when varying m (features) on "Pumadyn-8nh"

Figure 10: Distribution on largest singular values for various m and n

4.3.3 Discussion

The Laplacian kernel only reaches singular values below 10^{-6} in the "Abalone" dataset when varying m. Therefore we don't have enough data to analyse this kernel.

When we are varying the number of data points, n, we see that the drop-off point does not shift by much when n increases, especially for large values of n. For the polynomial kernels the drop-off point does not shift at all in the case of "Pumadyn", as does the linear kernel for "Abalone".

When varying the number of features m, the drop-off point consistently shifts. Comparing "Abalone" and "Pumadyn", we see that only the RBF (and Laplacian) behave differently. The linear and second order kernel, on the other hand, show no visible differences for "Abalone" and "Pumadyn".

In conclusion, the drop-off point seems to be much more sensitive to the number of features m than to the number of data points n. There are some differences visible between the different datasets, but the differences between the different kernels is much more pronounced.

5 Randomized approach

As we have seen from the results in section 3, the construction of the kernel matrix K proved to be a severe hindrance in solving the Kernel Ridge Regression Problem. We find that to reach sufficient levels of accuracy in our regression predictions, we seek a relatively large training set. However, this in turn spikes our computational runtime for solving the regression problem; the nature of K limits our ability to access truly accurate predictions for the target class. Fortunately, this computational bottleneck can be resolved by exploring the rank qualities of the kernel matrices. In section 4 we showed how a notably low percentage of the singular values of K were of orders $> 10^2$, while most of the remaining values were of orders $< 10^{-9}$. This suggests that most of the information gathered in the construction of K lies in a small proportion of its columns.

To this end, we introduce a method for approximating the kernel matrix: we seek a method that preserves both the semi positive definite nature of K and it being a Gram matrix. We also choose an approximation method that is as interpretable as K, meaning that its columns still correspond to the actual data that would be used in the construction of K. Following the work of [Alaoui and Mahoney, 2015], we will introduce the *Nyström approximation of the kernel matrix* K.

5.1 Theory: Nyström approximations

Let $K \in \mathbb{R}^{n\times n}$, $S \in \mathbb{R}^{n\times s}$ be a sketching matrix where $s \ll n$. Furthermore, the columns of S are standard unit vectors e_i with $i = 1, ..., n$ being the index of the column that is to be sampled. The sampling of s columns of K can now be defined as

$$C := KS.$$

Note that constructing C does not actually require us to perform a matrix multiplication, we can just simply compute and copy the sampled columns of K only. For the sampling of the columns we can choose whether or not we allow columns to be picked multiple times (i.e. with or without replacement). Given the above definition, allowing for repeated sampling, would cause S (and consequently C) to not be full-rank, whereas sampling without repeating columns, would guarantee that they are. Therefore we choose to sample the columns without replacement going forward. Since K is symmetric, sampling the rows at the same indices as the columns, yields the transpose of the same matrix:

$$R := S^\top K = S^\top K^\top = (KS)^\top = C^\top.$$

In addition, we introduce $W := S^\top KS$. Due to the symmetry of K, this is simply the intersection of the sampled rows and columns in K. Note, again, that constructing W does not actually require us to perform matrix multiplications, we just need to compute and copy the required entries of K.

Now we are ready to introduce the Nyström approximation of the kernel matrix K, which is defined as:

$$\hat{K} := C(S^\top KS)^\dagger R = CW^\dagger C^\top. \tag{9}$$

It is worth noting that $\hat{K}$ is of the same dimensions as K $(n \times n)$ and is low-rank. In fact, it is at most rank s. We can also verify that if we let $s = n$ and $S = I_s$, then $\hat{K} = CW^\dagger C^\top = KS(S^\top KS)^\dagger S^\top K = KI(I^\top KI)^\dagger I^\top K = KK^\dagger K = K$. That is, if we approximate K by sampling all of its columns, then we arrive back to K.

Computing $\hat{K}$

As for the implementation of the Nyström approximation, we will refer to **Algorithm 5.6** *NysSVD* used in [Tropp and Webber, 2023], see appendix for the precise algorithm as we implemented it. In short, this algorithm aims to form $\hat{K}$ as in definition (9) but avoids forming $W^\dagger$ explicitly and instead opts for an orthogonal - diagonal - orthogonal factorisation

$$\hat{K} \approx U\Lambda U^\top \tag{10}$$

which is both numerically stable and efficient for kernel ridge regression. This is achieved through the use of a stabilised Cholesky decomposition of W. The choice of inverting triangular matrices here rather than W keeps the computational cost low for computing the Nyström approximation of K.

Solving the Linear System

Now that we have established our method for computing our approximated kernel matrix, we will now introduce how we will implement this to solve our linear system for $\hat{\alpha}$ as in equation (6). Directly computing $(\hat{K}+n\lambda I)^{-1}$ is of the order $O(n^3)$, which would defeat the purpose of using Nyström. Instead, we can look to use an identity that exploits the low-rank approximation to $\hat{K}$ as in equation (10). To this end, we look to the *Woodbury Identity* for inverting conformable matrices A, U, C and V:

$$(A+UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1}+VA^{-1}U)^{-1}VA^{-1}. \tag{11}$$

This identity allows us to replace the inversion of a large $n \times n$ matrix with the inversion of a much smaller $s \times s$ one, provided that C has low rank. If we set $A = n\lambda I$, $UCV = U\Lambda U^\top \approx \hat{K}$, we will arrive at the following identity:

$$(\hat{K}+n\lambda I)^{-1} = \frac{1}{n\lambda}I - \frac{1}{(n\lambda)^2}U\left(\Lambda^{-1}+\frac{1}{n\lambda}I\right)^{-1}U^\top. \tag{12}$$

$VA^{-1}U$ reduces to $\frac{1}{n\lambda}I$ since U is orthonormal - $U^\top U = I$. Λ is diagonal, so its inverse is trivial. Moreover, we can see that the only remaining inversion to be computed is drastically simplified. The matrix

$$\Lambda^{-1}+\frac{1}{n\lambda}I$$

is diagonal as well, and its inverse can therefore be computed entry wise:

$$\left(\Lambda^{-1}+\frac{1}{n\lambda}I\right)^{-1} = \begin{bmatrix} \frac{1}{\frac{1}{\Lambda_1}+\frac{1}{n\lambda}} & & & \\ & \frac{1}{\frac{1}{\Lambda_2}+\frac{1}{n\lambda}} & & \\ & & \ddots & \\ & & & \frac{1}{\frac{1}{\Lambda_s}+\frac{1}{n\lambda}} \end{bmatrix}. \tag{13}$$

In other words, this inversion reduces to s scalar operations, one for each eigenvalue of the Nyström approximation. This means that overall, the solution to our linear system in equation (6) requires only a handful of matrix-vector multiplications with U, making the overall complexity of this method scale linearly with n as opposed to n^3.

In summary, we now have a computationally feasible method for approaching the kernel ridge regression problem by using the Nyström approximation sampling method and the Woodbury matrix identity in unison.

5.2 Experiment: Performance of Nyström Approximations, Uniform Sampling

Having introduced the Nyström approximation, we now investigate how well it approximates the kernel matrix K and how this improves as we increase the number of sampled columns. For now we stick to sampling these columns according to a uniform distribution.

5.2.1 Experimental set-up

We use the same three data sets as before, only now with a fixed kernel, λ and number of training data n:

- **Abalone:** RBF kernel, $c = 0.1$, $\lambda = 10^{-6}$, $n = 2000$
- **Pumadyn-8fm:** 2nd order polynomial kernel, $c = 0.1$, $\lambda = 10^{-5}$, $n = 2000$
- **Pumadyn-8nh:** 3rd order polynomial kernel, $c = 0.1$, $\lambda = 10^{-5}$, $n = 2000$

These specific configurations where chosen such that they provide a good regression performance on the specific data set, according to the results in the experiment in section 3.2.

For each data set, we vary the sampling fraction $\varepsilon \in [0.001, 0.1]$, corresponding to sampling

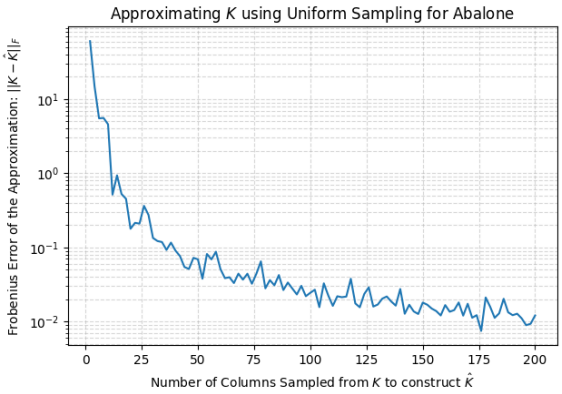
$$s = \varepsilon n$$

columns uniformly without replacement. For each s, we construct the Nyström approximation $\hat{K}$ using the sampled columns. The implemented steps are:

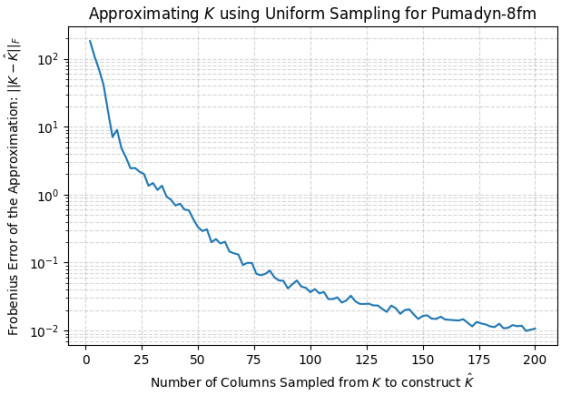
1. Sampling column indices uniformly from $\{1, \ldots, n\}$
2. Constructing the matrices C and W from the selected columns
3. Computing the exact kernel matrix K as well as the Nyström approximation $\hat{K} = U\Lambda U^\top$ using the *NysSVD* algorithm as described earlier
4. Measuring the accuracy of the approximation using the Frobenius error $||K - \hat{K}||_F$

This process is repeated for 100 values of ε, producing an error curve that shows how our approximation improves as the number of columns sampled increases.

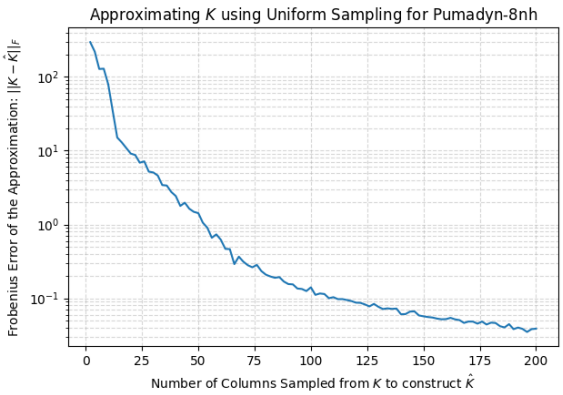
5.2.2 Results



(a) Frobenius error of $\hat{K}$ for the Abalone data set



(b) Frobenius error of $\hat{K}$ for the Pumadyn-8fm data set



(c) Frobenius error of $\hat{K}$ for the Pumadyn-8nh data set

Figure 11: Approximation errors for 3 data sets using uniform sampling of columns from K.

5.2.3 Discussion

In figure (11a), the Frobenius error decreases overall as the sample size s increases, but the curve is noticeably more irregular and oscillatory compared to the Pumadyn data sets. This could be due to some dense clusterings of data being present in the abalone data set which, if not proportionally represented in the column selection, may result in switches in the error direction and thus a more erratic descent. What also may play a factor here is our removal of duplicates in pre-processing. This increases the variability of the kernel matrix since all columns/rows of K have to be unique. Uniform sampling then might be more prone to disproportionally representing noisy/sparse data, just as it may not accurately describe the aforementioned dense clusterings. The RBF kernel is known to be more sensitive than polynomial kernels to variations in data points (exponential vs. algebraic) and as such, causes fluctuations in the accuracy of the approximation if it does not accurately represent the data clusters. With all of this said, we still see a relatively low error in our approximation of the kernel matrix when only 200 columns of K are sampled.

From figure (11b), we see a smooth and linear decrease in error as the number of sampled columns increases. This would imply that we have an accurate choice of kernel and that the kernel matrix was well approximated by a relatively moderate sample size. The choice of second order polynomial kernel captures the underlying relationships in the data effectively and as such, we do not require much of K to represent this.

In contrast, figure (11c) shows significantly larger Frobenius error across all sample sizes, and the approximation improves much slower than for 8fm. The nonlinearity documented in this data set could be a reason for this; the more important directions of the kernel matrix may not be evenly distributed across the columns, so uniform sampling cannot represent this accurately. Perhaps our choice of kernel is not the most optimal fit for this data set, or it could be that this data set requires more sophisticated sampling techniques to accurately represent the kernel matrix.

6 Informed column sampling

In the previous section, we sampled the columns of K according to a uniform distribution, but let us recall what a specific column of K represents: the jth column of K shows us how aligned the data point x_j is to every other data point in the data set. It should thus reveal how influential x_j is in obtaining an accurate predictor.

So, instead of uniform column sampling, we could sample the columns of K according to how important we suspect them to be. To this end, we need to give each column j a score (denoted by ℓ_j) and we then sample the columns with probabilities p_j according to these scores. In this section, we introduce four different approaches for computing such scores for sampling columns of the kernel matrix K and perform experiments to test their performance.

6.1 Theory: Column scoring approaches

Diagonal Scores

This method is quite straightforward; we will sample columns of K based on a probability distribution formed from the diagonal entries K_{jj} of the kernel matrix. That is, we sample the j-th column of the kernel matrix with probability

$$p_j = \frac{K_{jj}}{\sum_{i=1}^{n} K_{ii}}.$$

The advantages of this method are that it only requires the diagonal of K to be explicitly calculated, keeping the computational cost close to that of the uniform sampling method while still accessing some information inside of K. Each diagonal entry K_{jj} represents how influential the point x_j is after being mapped through the kernel. So, larger diagonal entries correspond to data vectors in the features with a greater RKHS norm and therefore contribute more strongly to the structure of the kernel matrix. This means that sampling columns of K using the above probability distribution biases the selection towards points that have a greater influence on the overall geometry of K.

In summary, this method provides a simple and low-cost alternative to more sophisticated sampling techniques that we will introduce later on. It retains the computational simplicity of uniform sampling while incorporating a small amount of structural information from the kernel matrix. However, it is worth noting that

Consider for example the for example the RBF kernel function: $K_{jj} = K(x_j, x_j) = e^{-c\|x_j - x_j\|_2} = e^{-c*0} = e^0 = 1$. Therefore each diagonal is equally large and the resulting probability distribution is simply a uniform distribution. Similarly, for relatively small c (in this report we use $c = 0.1$) and a small feature range ($[0, 1]$ after normalization, as used in this report), the polynomial kernel diagonals are also all approximately equal to 1: $K_{jj} = K(x_j, x_j) = (1 + 0.1x_j^\top x_j)^d = (1 + 0.1r)^d$ with $0 \leq r <\leq 1$. This means that the value on the diagonal equals at least 1^d and at most 1.1^d. All these properties of specific kernel functions and parameter choices could thus severely hinder the performance of the diagonal sampling method.

Statistical Leverage Scores

We have so far seen 2 column sampling methods which in theory should provide us with accurate approximations to K. With that said, we know that there is far more information about the kernel matrix that these methods do not quite capture. In many cases, the most informative columns are not those with large diagonal entries, but those that best span the dominant subspace of K. To this end, we will now introduce the statistical leverage scores of the kernel matrix K.

Let $K = U\Lambda U^\top$ be the eigenvalue decomposition of $K \in \mathbb{R}^{n\times n}$ and let $U_k \in \mathbb{R}^{n\times k}$ denote the matrix consisting of the top k eigenvectors of K. Then the statistical leverage score of the j-th column of K is defined as

$$\ell_j = ||(U_k)_j||_2^2. \tag{14}$$

The parameter $k \in \{1, \ldots, n\}$ specifies the **target rank of the approximation** $\hat{K}$ which is given by $U_k \Sigma_k U_k^\top$. Intuitively, this means that the statistical leverage scores measure how much a specific column

of K contributes to this dominant k-dimensional subspace. So, if the leverage score ℓ_j is relatively large, then x_j plays an important role in representing the top-k structure of the kernel matrix.

Suppose we now want to implement this method - we sample s columns of K using a probability distribution

$$p = (p_1, \ldots, p_n), \qquad \text{with } p_i = \frac{\ell_i}{\sum_{j=1}^{n} \ell_j},$$

with ℓ_i defined previously. This would require computing all of K, which is what we are trying to avoid when using Nyström approximations, so in practice an approximate leverage score probability distribution is constructed with $p_i \geq \frac{\beta}{k}\ell_i,\ \beta \in (0,1]$. Defining $\hat{K} = CW^{\dagger}C^{\top}$ as before and $\varepsilon \in (0,1]$ some approximation factor, lemma 5 from [Gittens and Mahoney, 2016] shows that if we sample

$$s = O\left(\frac{k\log(k/\beta)}{\beta\,\varepsilon^2}\right)$$

columns according to a β-approximate leverage score distribution, then with high probability, the Nyström approximation above achieves the guarantees

$$\|K - \hat{K}\|_2 \;\leq\; \|K - K_k\|_2 \;+\; \varepsilon\,\|K - K_k\|_*,$$

where K_k is the best rank–k approximation to K and $|\cdot|_*$ is the nuclear norm of the matrix.

This tells us that the sample size s scales only as $O(k\log k)$ meaning that we will not need much of K to retain a high level of accuracy. Furthermore, we see that the approximation error is close to the optimal rank–k error and that even approximate leverage scores are sufficient for our construction of $\hat{K}$. In summary, this result explains why leverage–score sampling is the preferred strategy for the Nyström approximation, it provides strong theoretical guarantees while requiring only a small number of sampled columns. This will lead us into exploring other leverage scoring techniques.

λ-Ridge Leverage Scores

We will now look to further improve our sampling methods by exploring a λ-dependent analogue to the statistical leverage scores called the λ-ridge leverage scores, introduced by [Alaoui and Mahoney, 2015]. Given the usual definitions of the kernel matrix $K \in \mathbb{R}^{n\times n}$, its eigenvalue decomposition $K = U\Lambda U^{\top}$ and the regularisation parameter λ from equation (2), the **λ-ridge leverage scores** are defined as follows:

$$\ell_j(\lambda) = \sum_{i=1}^{n} \frac{\sigma_i}{\sigma_i + n\lambda} U_{ij}^2, \qquad \text{for all } j \in \{1, \ldots, n\}. \tag{15}$$

By using the eigenvalue decomposition of K, the orthogonality property of U and that the product of any two diagonal matrices is also a diagonal matrix, we can show the equivalence

$$\ell_j(\lambda) = [K(K + n\lambda I)^{-1}]_{jj}.$$

This result is quite significant; it tells us that the λ-ridge leverage score of data point x_j **is exactly how much** that data point will affect its own predicted value in the Kernel Ridge Regression problem.

Lastly, we note that the sum of the λ-ridge leverage scores gives us **the effective dimension** of the learning problem,

$$\sum_{j=1}^{n} \ell_j(\lambda) = Tr\left(K(K + n\lambda I)^{-1}\right) = d_{\text{eff}}(\lambda).$$

In other words, this sum measures the intrinsic complexity of the KRR problem and plays a central role in the statistical guarantees of the Nyström method. In particular, the number of columns needed for an accurate Nyström approximation scales with $d_{\text{eff}}(\lambda)$ and not with n directly. Furthermore, theorem 3 from [Alaoui and Mahoney, 2015] proves that if the columns of K are sampled with probabilities proportional to $\ell_j(\lambda)$, then a Nyström approximation using only $p \sim O(d_{\text{eff}}(\lambda))$ columns achieves essentially the same statistical risk as using the full kernel matrix K.

However, there is one major downside to the sampling method $\ell_j(\lambda) = [K(K + n\lambda I)^{-1}]_{jj}$, which is that it still requires the complete construction of K or its eigenvalue decomposition, both of which are of order $O(n^2)$. This implies that the implementation of this method is as computationally expensive as solving the original Kernel Ridge Regression problem. For this reason, we will look to a method for approximating these scores without explicitly computing the kernel matrix K.

Approximate λ-Ridge Leverage Scores

We will now introduce what is arguably the most theoretically powerful sampling technique: the **approximate λ-ridge leverage scores** of [Alaoui and Mahoney, 2015]. The key idea here is to sample only a small subset $p \ll n$ columns of K. The computations for computing these approximate λ-ridge leverage scores are then performed in this reduced p-dimensional space. A detailed description of this algorithm is provided in the appendix. The overall runtime for this is $O(np^2)$ for $p \ll n$. This is a massive improvement over the previous $O(n^3)$ for constructing all of K and computing the exact leverage scores. Furthermore, theorem 4 from [Alaoui and Mahoney, 2015] shows us that if the probability distribution used for column sampling is defined by the diagonal of the kernel matrix $p_i = K_{ii}/Tr(K)$, then every approximate λ-ridge leverage score $\tilde{\ell}_i(\lambda)$ is a close multiplicative approximation to the true value of $\ell_i(\lambda)$. This guarantee allows us to still retain all the aforementioned advantages of using λ-ridge leverage scores over the statistical leverage scores, but without the computational overhead. We can accurately prioritise the most informative columns of K without ever forming K.

Lastly, it is important to note here that out of the 5 sampling techniques introduced, both the statistical leverage scores and the exact λ-ridge leverage scores require the formation of the entire kernel matrix K, which will, of course, defeat the purpose of opting for a Nyström approximation in the first place. We therefore will only consider the implementation of the uniform sampling, diagonal entries score sampling, and the approximate λ-ridge leverage scores sampling going forward.

6.2 Experiment: Parameter Selection for Approximate λ-Ridge Leverage Scores

As seen previously, the approximate λ-ridge leverage scores require a sampling of $p \ll n$ columns from K. Firstly, it is worth clarifying that this sampling parameter is only to be used for constructing the leverage scores and is not related to the sampling parameter of ε used to construct $\hat{K}$ as seen in the section on Nyström approximations. Secondly, we will stick to the column sampling probability distribution formed using **the diagonal of** K, as demonstrated by [Alaoui and Mahoney, 2015]. This choice keeps both the computational cost very low while also providing accuracy guarantees as seen in their theorem 4 previously mentioned. This implies that the implementation of this scoring technique will only construct the diagonal of K and its p sampled columns.

With all of the above said, we now investigate how large the value of p needs to be to provide a good kernel matrix approximations for the approximate λ-ridge leverage scores. Intuitively, larger values of p should give more accurate approximations, but they also increase the computational burden. We therefore aim to find the smallest value of p which still results in a good approximation to K.

6.2.1 Experimental set-up

For this experiment we keep the same combinations of data sets, kernel functions, and parameters used in section 5.2. For each data set, we construct the approximate λ-ridge leverage scores by implementing the algorithm as described in [Alaoui and Mahoney, 2015], using different values of the sampling parameter

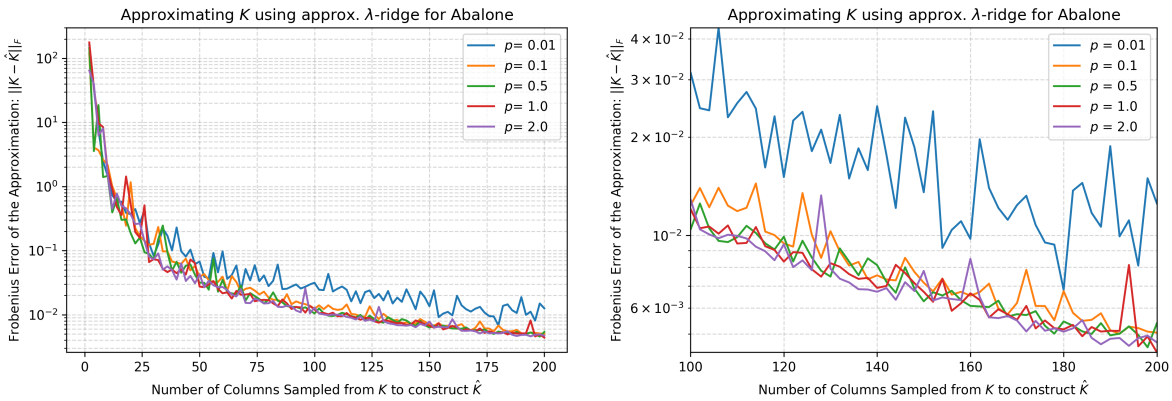
$$p \in \{0.5s,\ 1.0s,\ 1.5s,\ 2.0s,\ 3.0s\},$$

where $s = \varepsilon n$ represents the number of sampled columns of K. As before, the sampling fraction ε is varied in the interval

$$\varepsilon \in [0.001,\ 0.1].$$

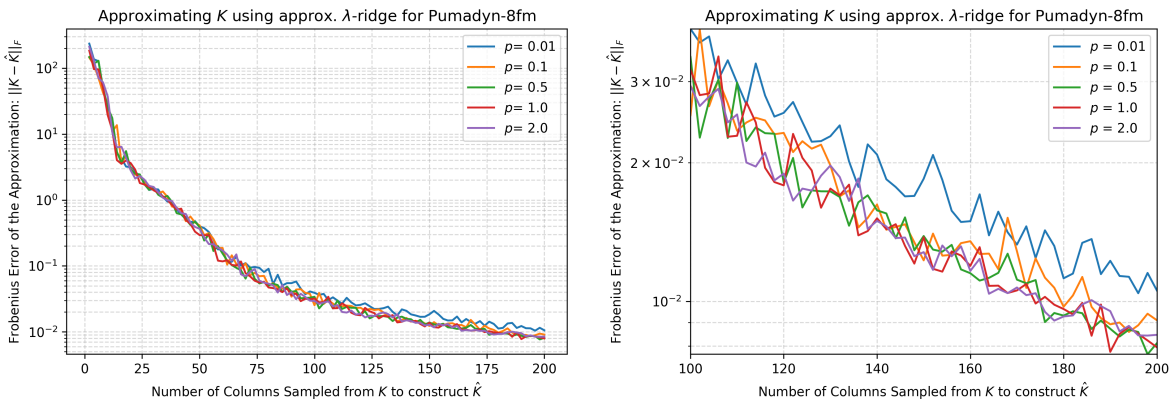
For each choice of ε and each value of p, we compute the approximate kernel matrix $\hat{K}$ using the Nyström approximation with columns chosen according to the λ-ridge leverage scores distribution. For every data set, five curves, one for each value of p, is plotted with the number of sampled columns against the Frobenius approximation error $||K - \hat{K}||_F$.

6.2.2 Results



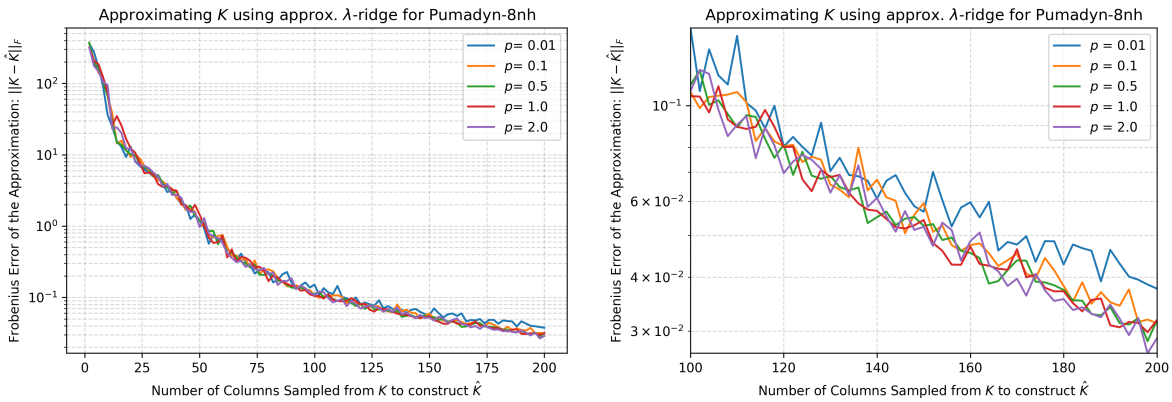
(a) Errors in approximating K for various s (b) Errors in approximating K for various s (zoomed)

Figure 12: Experiment results for the Abalone data set (RBF, $c = 0.1$, $\lambda = 10^{-6}$, $n = 2000$)



(a) Errors in approximating K for various s (b) Errors in approximating K for various s (zoomed)

Figure 13: Experiment results for the Pumadyn-8fm data set (2nd order, $c = 0.1$, $\lambda = 10^{-5}$, $n = 2000$)



(a) Errors in approximating K for various s (b) Errors in approximating K for various s (zoomed)

Figure 14: Experiment results for the Pumadyn-8nh data set (3rd order, $c = 0.1$, $\lambda = 10^{-5}$, $n = 2000$)

6.2.3 Discussion

Starting with the Abalone data set in figure 12, all values of p exhibit the expected trend where the approximation error decreases smoothly as the number of sampled columns increases. From the zoomed in plot, it is clear that $p = 0.01$ performs quite poorly compared to the rest of the values. We can see that the moderate values of p (0.1, 0.5) show strong performance while the larger values of p (1.0, 2.0) offer only marginal improvements. This would lead us to believe that the approximation to K is close to the optimal value for some value of $p \approx 0.1$. These diminishing returns from larger values of p is evident in the zoomed in plot where all curves but the blue curve are tightly clustered with slight spontaneous irregularities in all curves.

In figure 13, we see a similar result where all values of p show decreasing error as $\hat{K}$ approaches K and there is overall less variance in the curves compared to that of Abalone. Once again, $p = 0.01$ performs much worse than the other four curves and even by $p = 0.1$, there are drastic improvements in the Frobenius error.

Lastly, in figure 14, the error curves follow a similar decaying shape, but the overall error levels are noticeably higher. This reflects the greater complexity of the Pumadyn-8nh kernel matrix. The effect of the sampling factor p is more pronounced comparing to the previous plots, smaller values of p produce overall more noisy curves while increasing p leads to noticeably smoother and lower-valued curves. This suggests that larger values of p could be necessary to sufficiently capture the complexity of this data set and its corresponding matrix.

6.3 Experiment: Comparison of Informed and Uninformed Column Sampling

The goal of this experiment is to investigate whether the aforementioned, informed column sampling approaches provide more accurate approximations than the uninformed, uniform sampling.

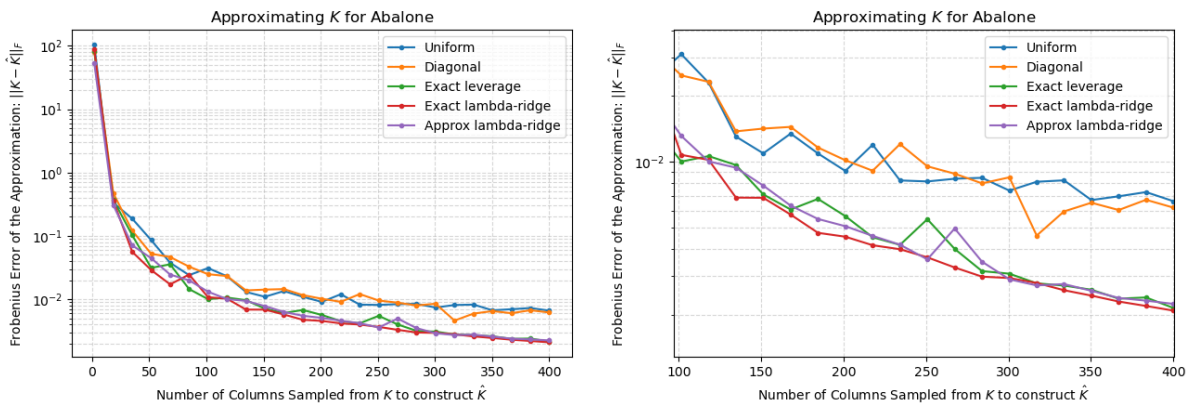
6.3.1 Experimental set-up

The same combinations of data sets, kernel functions, and parameters from section 5.2 are used. The considered sampling approaches are uniform sampling, diagonal scoring, exact statistical leverage scoring (with $k = s$), exact λ-ridge leverage scoring, and approximate λ-ridge leverage scoring (with $p = 0.1s$).

For each data set, we vary the sampling fraction $\epsilon \in [0.001, 0.2]$, corresponding to sampling $s = \epsilon n$ columns according to the considered sampling approach.

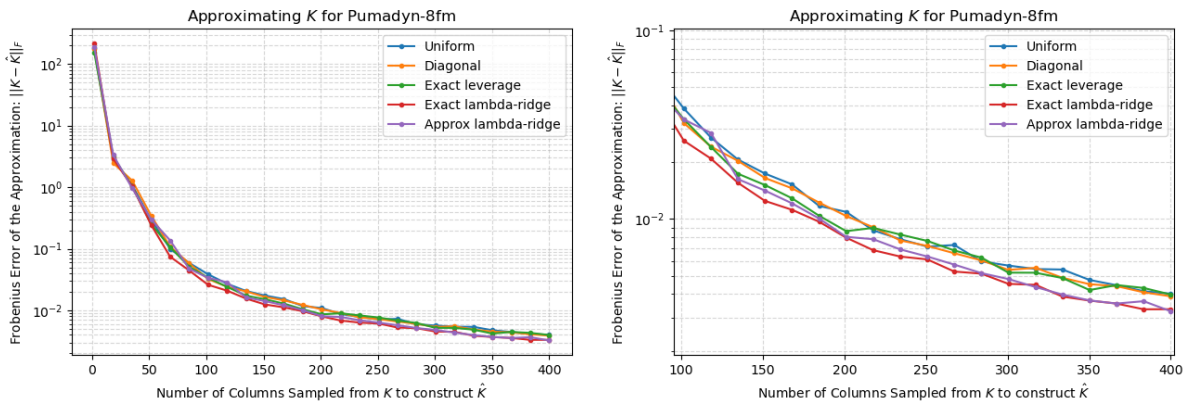
Per data set, this results in 5 different plots (for the sampling approaches) with on the x-axis the number of sampled columns (s) and on the y-axis the Frobenius error: $\|K - \hat{K}\|_F$.

6.3.2 Results



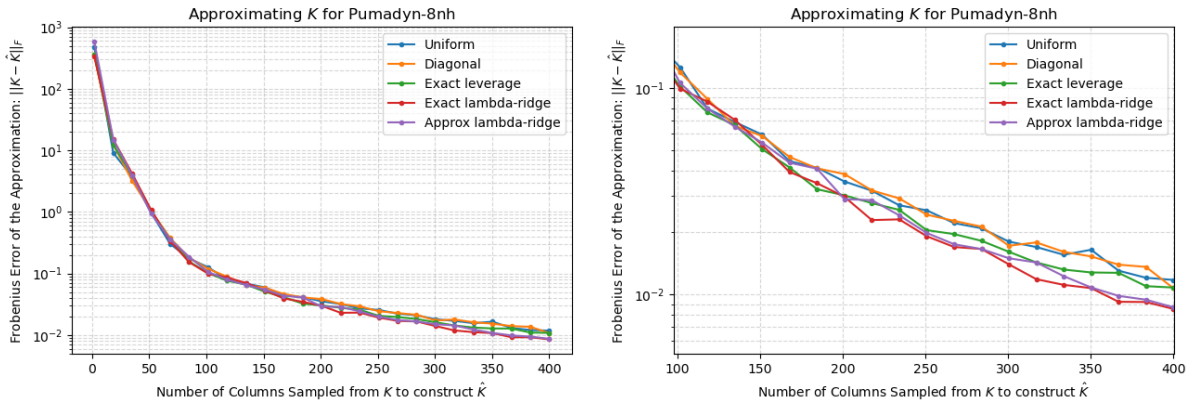
(a) Errors in approximating K for varying s (b) Errors in approximating K for varying s (zoomed)

Figure 15: Experiment results for the Abalone data set (RBF, $c = 0.1$, $\lambda = 10^{-6}$, $n = 2000$)



(a) Errors in approximating K for varying s (b) Errors in approximating K for varying s (zoomed)

Figure 16: Experiment results for the Pumadyn-8fm data set (2nd order, $c = 0.1$, $\lambda = 10^{-5}$, $n = 2000$)



(a) Errors in approximating K for varying s (b) Errors in approximating K for varying s (zoomed)

Figure 17: Experiment results for the Pumadyn-8nh data set (3rd order, $c = 0.1$, $\lambda = 10^{-5}$, $n = 2000$)

6.3.3 Discussion

Looking at figure 15 (both plot a and b), we observe the kind of behaviour we would have expected. The uninformed, uniform sampling approach seems to consistently produce the worst approximations. As expected (see section 6.1) the diagonal scoring approach performs basically the same as uniform sampling. The approximate λ-ridge leverage scoring always performs somewhat worse than the exact λ-ridge leverage scoring, but both perform considerably better than the uniform and diagonal sampling approaches. Finally, the exact statistical leverage scoring approach performs very similar to that of the approximate and exact λ-ridge leverage scores.

Looking at plots 16a and 17a, it appears as though this pattern is not present for the Pumadyn data sets. When zooming in, however, the same pattern does in fact become visible (see plots 16b and 17b). The differences are just of a smaller scale than for the Abalone data set. The main deviation from the pattern described in the previous paragraph, is the performance of the exact statistical leverage scores, which is now seems to trend more towards the performance of the uniform sampling, rather than the λ-ridge leverage score approach.

When zooming in on the first part of the x-axes however (i.e. $s = 0$ to $s = 100$) this pattern seems to become less clear. These zoomed plots are not included in the results section, as it could be be captured well in a single frame. The overall behaviour for all three data sets is however the same: the smaller the sampled number of columns, the less clear the described performance pattern becomes. For small values of s, it appears to be completely random which of the five approaches performs best. The main difference between the Abalone and Pumadyan data sets, is that the expected pattern (as described in the previous paragraphs) starts to show already for a much smaller of value of s.

The fact that the described pattern (i.e. λ-ridge leverage scores outperform uniform sampling and diagonal scoring) seems to hold for all three data sets, suggests that the pattern is in fact general. It appears to holds true for any data set, but the extend of its impact may differ per data set.

Recalling that the scores are mainly beneficiary in unevenly distributed data sets (see [Alaoui and Mahoney, 2015]), this dependence on the specific data set, appears to make sense. The more evenly distributed a data set is, the less impact the informed scores can make and the less clear the general pattern is.

The fact that the pattern becomes more clear as s increases (and simultaneously is not clear/visible for very low values of s) should also not be surprising. Remember that we are sampling the columns according to a probability distribution based of the scores. Sampling only a few columns, can hardly be representative of the full distribution and is thus very unlikely to contain the 'full image' of the underlying scores. Only as we pick increasingly more columns, does the sampled distribution actually start to resemble the probability distribution.

6.4 Experiment: Performance of Viable Column Sampling Approaches

Let us recall the overarching aim of this report: to use a (randomized) low-rank approximation to K to solve the KRR minimization problem with similar accuracy to the classical approach, but at a lower computational cost. As concluded in section 3, one of the main bottlenecks in this regard, is having to compute the full kernel matrix K by repeated evaluations of the kernel function. With this is mind, we must conclude that the exact statistical leverage scores and the exact λ-ridge leverage scores are no viable options. In the computation of these scores, we namely need to use every element of the kernel matrix K, which defeats our purpose for using them in the first place.

We therefore narrow our focus to the other three sampling approaches, which do in fact only need a portion of the kernel matrix K to be computed. The goal of this experiment is to test the performance of the Nyström approximation with these three sampling approaches on regression data sets. We investigate both the regressions performance of the resulting predictor and the computational cost required to find this predictor.

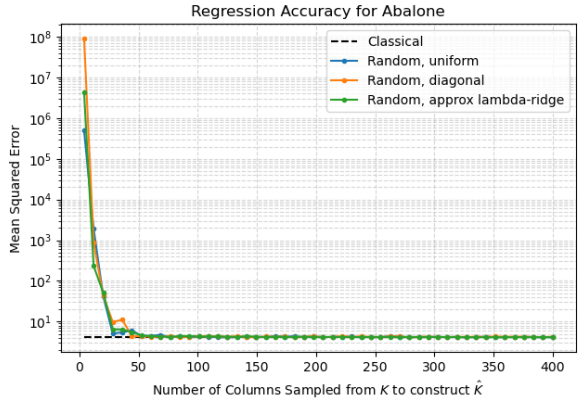
6.4.1 Experimental set-up

The same combinations of data sets, kernel functions and parameters from section 5.2 are used. The considered sampling scores are uniform sampling, diagonal score sampling, and approximate λ-ridge leverage score sampling (with $p = 0.1$).

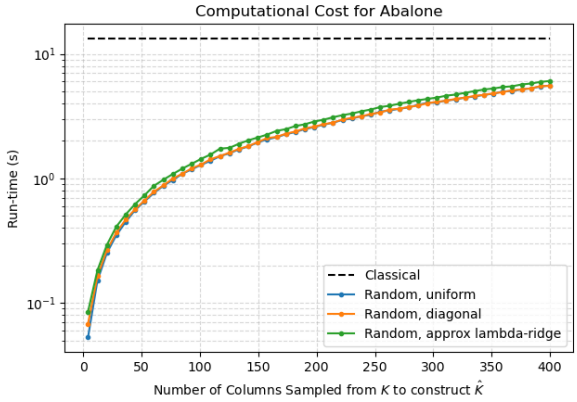
Since the value of n is fixed, we make one training set of n data points and one testing set of $n_{test} = 0.3n$ data points, for each data set. Then we vary the sampling fraction $\epsilon \in [0.002, 0.2]$ corresponding to sampling ϵn columns according to the considered sampling approach. For each epsilon and sampling approach, the predictor is tested on the testing set and its performance is measured using the mean squared error. Simultaneously the computational cost in terms of run-time is measured, using the Python package *time*.

Finally, also the classical approach (from section 3) is used to solve the KRR problem. Since this approach is independent from s (the number of sampled columns) it is only run once and the resulting accuracy and run-time are plotted as a straight line in the respective plots.

6.4.2 Results

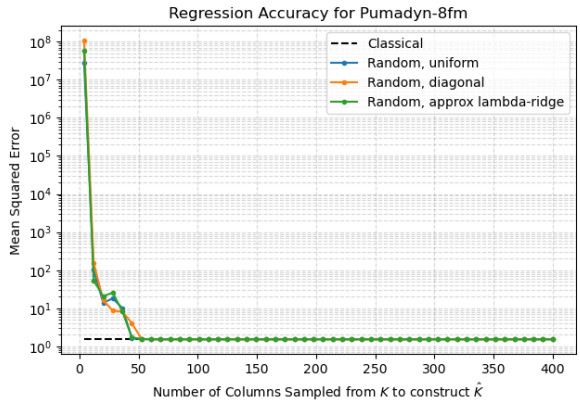


(a) Effect of s on error

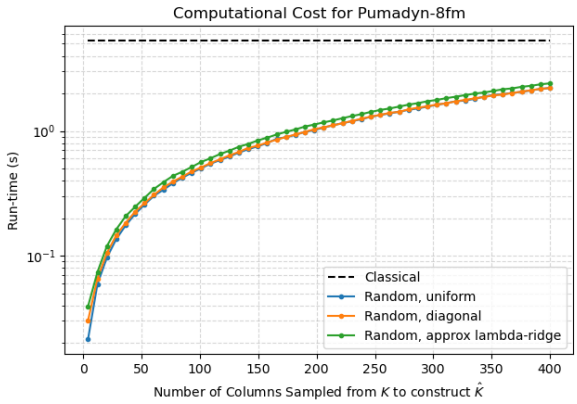


(b) Effect of s on run-time

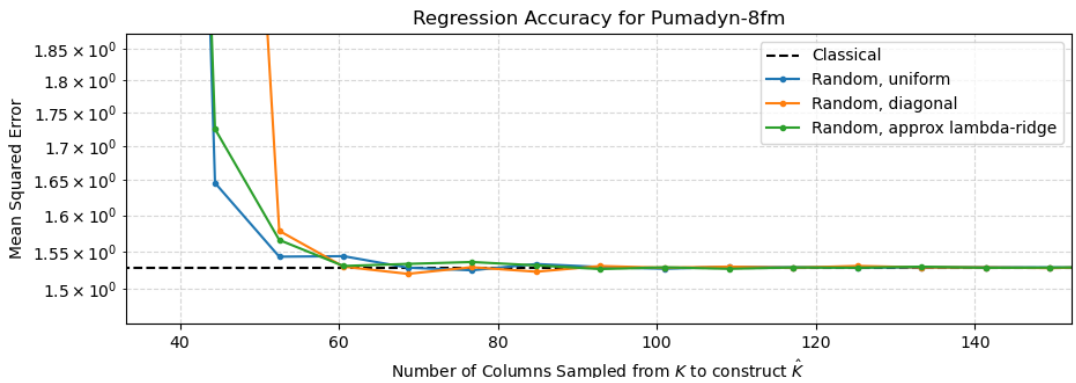
Figure 18: Experiment results for the Abalone data set (RBF, $c = 0.1$, $\lambda = 10^{-6}$, $n = 2000$)



(a) Effect of s on error

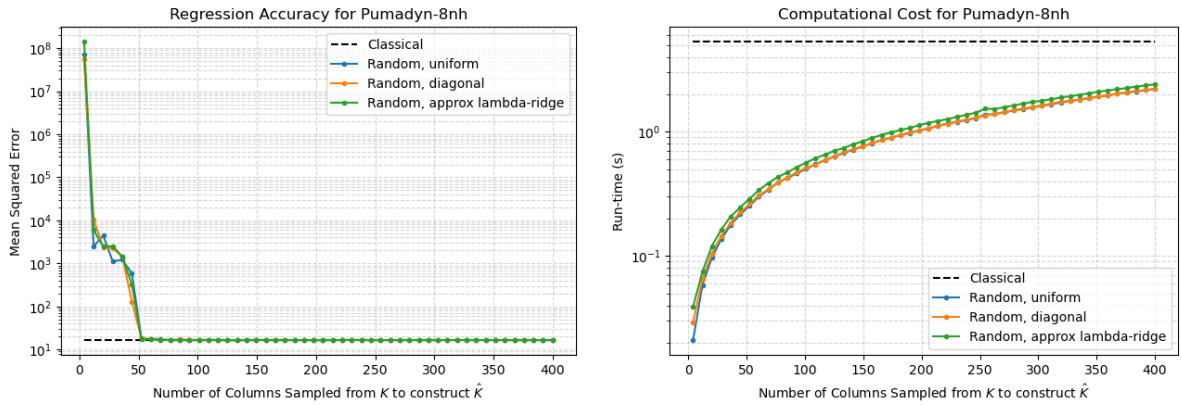


(b) Effect of s on run-time



(c) Effect of s on error (zoomed in)

Figure 19: Experiment results for the Pumadyn-8fm data set (2nd order, $c = 0.1$, $\lambda = 10^{-5}$, $n = 2000$)



(a) Effect of s on error

(b) Effect of s on run-time

Figure 20: Experiment results for the Pumadyn-8nh data set (3rd order, $c = 0.1$, $\lambda = 10^{-5}$, $n = 2000$)

6.4.3 Discussion

Accuracy

Looking at plots 18a, 19a and 20a, we see similar behaviour for all three sampling approaches as s increases for all three data sets. Namely, that the mean squared error of the randomized approach drops very quickly towards the error achieved by the classical approach. When zooming in (e.g. in plot 19c) we observe that even near this reference mean squared error this is no slow, asymptotic behaviour, but the reference value is really achieved that quickly. The precise speed of the drop appears to differ per kernel and data set, but clearly it is there. This corresponds well with the results from section 4, which showed that low-rank approximations to $\hat{K}$ do indeed still provide very good regression results, but that the precise behaviour also depends on the specific data set and kernel function.

Across all accuracy plots, there appears to be no trend in which sampling approach performs better: it seems rather random which approach results in the lowest mean squared error for each value of s. At first this seems surprising, as we would expect the informed methods (in particular the approximate λ-ridge) to be able to reach a lower error sooner than the uninformed, uniform sampling. Recalling our findings in section 6.3, however, we realize that the advantage of using an informed sampling only comes into effect for large enough values of s, due to the nature of probability distributions. Given the fact that the randomized approach already drops to a very good mean squared error for $n \approx 50$, it is thus entirely reasonable that the effects of the informed sampling approaches are not yet visible.

If we were to pick a value of s for which all considered approaches on all considered combinations of data sets and kernel function is guaranteed to perform basically just as well as the classical method, we would pick $s = 100$. In the context of this plot, this may seems rather on the high end, but considering that the total number of training data points is $n = 2000$, it still is quite a large reduction.

Computational cost

Looking at plots 18b, 19b and 20b, the run-time of the randomized approach obviously increases as s increases, but it takes quite a while to reach the same cost as classical approach. This overall behaviour is similar across all three plots, which is to be expected, as the run-time.

The only notable difference between the 3 plots, is the scale on the y-axis. Here we observe that solving the Abalone data set (with the RBF kernel) is more computationally than solving the Pumadyn data sets (with the 2nd and 3rd order kernels). This corresponds well with the results from section 3.3, where we concluded that using the RBF/Laplacian kernel function resulted in higher costs compared to the polynomial/linear kernel functions, but only by a constant factor. Hence the plots look similar, only shifted upwards for the Abalone data set with the RBF kernel function.

Across all three plots (18b, 19b and 20b), there is a clear distinction in how computationally expensive each of the three different sampling approaches are. The uniform sampling and diagonal scoring need an almost identical run-time, but upon very close inspection the uniform sampling approach is consistently

ever so slightly faster. The $\lambda-$ridge leverage scoring approach grows in the same fashion with s as the other two methods, but it is notably more expensive by a, seemingly constant.

Overall, the differences between the plots for the three different approaches are however not very large, which suggests that the differences in computational cost associated to computing the scores is relatively small. We would expect that the gaps between the three approaches becomes larger for even more computationally expensive kernel functions (or a higher number of data features (m)) because then the computation of the scores gets more expensive.

The fact that the cost of the approximate λ-ridge leverage scoring approach is so comparable to the other methods, is somewhat surprising, as computing these scores would appear to be a rather expensive task. We suspect that the choice of $p = 0.1s$ (see section 6.2) has a major effect in keeping the computational cost to a minimum, and overall we conclude that the implementation of this approach is rather efficient.

Given that we just concluded that $s = 100$ is a good choice in terms of prediction accuracy, it is interesting to look at the computational cost for $s = 100$. In all plots (18b, 19b and 20b), we observe that for at this point, the run-time of the randomized approach is a lot lower than that of the classical approach (approximately by a factor of 10). This means that this approach through Nyström approximations using (possibly informed) random sampling of the columns of K is indeed a good alternative to the classical approach. It reaches a similar level of performance in terms of regression accuracy, but at a considerable lower computational cost.

7 Conclusion

Having introduced the KRR minimization problem in section 2 as a framework for solving many types of optimisation problems, we have shown in section 3 that this approach yields good results on real-life regression data sets. We have introduced the cross-validation as an approach to prevent over-fitting or under-fitting and concluded that, to be as sure as possible on the performance of the predictor, the number of training data points (n) should taken as large as feasible.

At the same time, however, we also observed that the computational cost (in terms of run-time) of finding the predictor scales with n. One of the main bottlenecks in terms of computational cost, turned out to be time needed to construct the kernel matrix K, which scales like mn^2 for all considered kernel functions.

In order to potentially reduce this computational cost, we investigated the possibility of using low-rank approximations to K in section 4. This could namely allow use to use only a part of the elements of K, instead of needing to compute the full matrix. The experiments in this section indeed found a low-rank structure in the singular value decomposition of kernel matrices. Although the precise behaviour may depend on the specific data set, the kernel function, the number of training data points (n) and the number of data features (m), this low-rank structure appeared to be an inherent property of each kernel function.

Further experiments showed that this property can be effectively exploited by using low-rank approximations of K to solve the KRR minimization problem. The resulting generalization performance was basically identical to that when using the full kernel matrix, even for relatively small ranks (e.g. a rank-50 approximation of a rank 2000 matrix).

In order to efficiently compute low-rank approximations to K for which not all of K has to be computed, the randomized approach of Nyström approximations was introduced in section 5. Experiments showed that by randomly sampling a total of s columns (with uniform probability) from the kernel matrix K, a sufficiently good approximation of K can be made with $s \ll n$. When looking at the mean squared error on regression data sets, this was even more pronounced. Some specific data set and kernel configurations only needed to use approximately 25 columns, while still having basically the same generalization performance as when the full $n = 2000$ data points were used.

To potentially improve on the uninformed, uniform sampling of columns, several approaches for informed sampling were introduced in section 6. Experiments showed that, in general, the informed approaches resulted in better approximations, but also that this effect becomes increasingly more visible as the number of sampled columns (s) increases. This also means that, for very low values of s the informed approaches do not show a clear improvement over the uninformed approaches. Additionally, this effect also depends on how evenly distributed the data points are throughout the features space: the less evenly distributed, the larger the effect of using the informed approaches.

Finally, the three viable scoring approaches were implemented with the randomized approach for solving the KRR minimization problem and tested on regression data sets. This experiment showed that the randomized approach does indeed reach the same accuracy as the classical approach, but generally does so by sampling only 100 of the 2000 columns of K. Run-time measurements showed that for $s = 100$, the randomized approach was about ten times faster than the classical approach. This leads us to conclude that the randomized approach (Nyström approximations) was indeed able to majorly decrease the computational cost of solving the KRR minimization problem, while still maintaining the same accuracy performance as the classical approach.

No effects in terms of regression accuracy was observed for the different scoring methods, which was likely due to the randomized approach already reaching the desired accuracy for low values of s. In this setting (with kernel functions and $n = 2000$), there thus seems to be no incentive to use informed scoring approaches over the uninformed approach. Especially when taking into consideration the fact that the former comes at a higher computational cost. Only when the problem is of such nature that considerably more columns need to be sampled, or the data points are distributed very unevenly across the feature space, would the use of informed column sampling seem beneficial.

Limitations

There are a number of factors that could affect the validity of these conclusion.

First of all, the experiments supporting the conclusions were performed on specific data sets, which were

are regression type problems. It might very well be that the behaviour observed on these data sets, is not present for other regression data set or other (non-regression) types of data sets.

Secondly, not all possible kernel functions were considered in this report. The extend to which these results hold for all kernel functions is unclear and even the specific choices for the parameters c could affect the validity of the conclusions.

Lastly, the values of n and m were typically fixed at 2000 and 8 respectively. For different values, the results may very well be different and more research into the affect of these parameters would be needed to substantiate or improve these conclusions.

References

[Alaoui and Mahoney, 2015] Alaoui, A. and Mahoney, M. W. (2015). Fast randomized kernel ridge regression with statistical guarantees. *Advances in neural information processing systems*, 28.

[Bunch and Kaufman, 1977] Bunch, J. R. and Kaufman, L. (1977). Some stable methods for calculating inertia and solving symmetric linear systems. *Mathematics of computation*, pages 163–179.

[El Khatib and Alkhatib, 2024] El Khatib, O. and Alkhatib, N. (2024). A comprehensive overview of kernels in machine learning: Mathematical foundations and applications. *International Journal of Computer (IJC)*, 53(1):150–172.

[Gittens and Mahoney, 2016] Gittens, A. and Mahoney, M. W. (2016). Revisiting the nyström method for improved large-scale machine learning. *Journal of Machine Learning Research*.

[Higham, 1997] Higham, N. J. (1997). Stability of the diagonal pivoting method with partial pivoting. *SIAM Journal on Matrix Analysis and Applications*, 18(1):52–65.

[Kaw et al., 2009] Kaw, A. K., Kalu, E. K., and Nguyen, D. (2009). *Numerical methods with applications*. University of South Florida.

[Tropp and Webber, 2023] Tropp, J. A. and Webber, R. J. (2023). Randomized algorithms for low-rank matrix approximations: Design, analysis and applications. *SIAM Journal on Scientific Computing*.

[Zhu, 2025] Zhu, R. (2025). Statistical machine learning in r.

8 Appendix

8.1 Data sets

All data sets were formatted so that each column is a feature, instances are rows and the target class is the final column of the file. All entries are to be separated by a blank space.

8.1.1 Abalone - https://archive.ics.uci.edu/dataset/1/abalone

A single `.data` file in `csv` format, consisting of 8 features and 4177 instances. The target class (integer) is the **number of rings** that appear on the abalone's shell, which can be used to calculate their age. All features were continuous data (length, diameter, whole weight, shell weight etc.) except for the first feature sex - M,F or I (infant). We edited this categorical data to be trinary (0-M, 1-F, 2-I). Once again, commas were replaced with blank spaces and the target class was kept as the last column.

8.1.2 Pumadyn - http://www.cs.toronto.edu/~delve/data/pumadyn/desc.html

The Pumadyn family consists of 8 data sets, each with 8192 instances and 8 or 32 features. These files vary in linearity of the data as well as levels of noise in the data per set. These data sets represent a realistic simulation of the dynamics of a Puma 560 robot arm. The task in these datasets is to predict the angular acceleration of one of the robot arm's links. Inputs include angular positions, velocities and torques of the robot arm. In particular, we use **Pumadyn-8fm** and **Pumadyn-8nh**. Note for the file names:

- **8** - Total number of features
- **f** or **n** - **f**airly linear or **n**onlinear data to be expected
- **m** or **h** - **m**oderate or **h**igh noise in the target class

8.1.3 Random data sets

Two additional data sets were randomly generated in the following manner. A total of n data points were generated, each existing of m features and a single target value (y). The exact values of the features and target values, were sampled from some probability distribution. In our case, this was a uniform distribution (betwoon 0 and 1) and a normal distribution (with mean = 0 and standard deviation = 1).

8.1.4 Preprocessing

Before we start using the data sets to construct predictors, the data sets are first checked for duplicates and normalized. Duplicate data points are removed as they provide no additional data and could lead to a singular kernel matrix K. The normalization is a so-called min-max scaling, which transforms all the data features to the interval $[0, 1]$.

To prevent so-called data leakage, the normalization only happens after the splitting into a training and test set. If we were to split the data before normalization, the training data would, through the scaling (which also inclused the testing data), implictly have received some information on the scale, mean, and/or variance of the testing data,

8.2 Computational Cost Analysis

8.2.1 $LDL^\top$ approach

In this section, we work out the computational cost in terms of flops of the $LDL^\top$ approach as described in section 3.1.1. We first recall the relevant construction formulas (7) and solving procedure (8):

$$d_{jj} = a_{jj} - \sum_{k=1}^{j-1} l_{jk}^2 d_{kk}$$
$$l_{ij} = \frac{1}{d_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik} d_{kk} l_{jk} \right)$$

1) Find z such that $Lz = b$. $\begin{bmatrix} 1 & 0 & \dots & 0 \\ l_{21} & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ l_{n1} & l_{n2} & \dots & 1 \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$ $\quad z_i = b_i - \sum_{k=1}^{i-1} l_{ik} z_k \quad i = 1, ..., n$

2) Find y such that $Dy = z$: $\begin{bmatrix} d_{11} & 0 & \dots & 0 \\ 0 & d_{22} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & d_{nn} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_n \end{bmatrix}$ $\quad y_i = \frac{z_i}{d_{ii}} \quad i = 1, ..., n$

3) Find x such that $L^\top x = y$: $\begin{bmatrix} 1 & l_{21} & \dots & l_{n1} \\ 0 & 1 & \dots & l_{n2} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$ $\quad x_i = y_i - \sum_{k=i+1}^{n} l_{ki} x_k \quad i = n, ..., 1$

Note that this procedure computes the products $d_{kk} l_{jk}$ for each d_{jj} as well as for each l_{ij}. An efficient implementation would only compute these product once, hence we consider these products as already computed when counting the number of flops for constructing L.

Constructing D:

$$\begin{aligned} \#\text{flops for } d_{jj} &= 1 \text{ subtraction} + (j-2) \text{ additions} + 2 * (j-1) \text{ multiplications} \\ &= 1 + j - 2 + 2j - 2 = 3j - 3 \end{aligned}$$

$$\#\text{flops for } D = \sum_{j=1}^{n} \#\text{flops for } d_{jj} = \sum_{j=1}^{n} (3j-3) = 3 \left(\sum_{j=1}^{n} j \right) - 3n = 3 * \frac{n(n+1)}{2} - 3n = \frac{3}{2}n^2 - \frac{3}{2}n$$

Constructing L:

$$\begin{aligned} \#\text{flops for } l_{ij} &= 1 \text{ division} + 1 \text{ subtraction} + (j-2) \text{ additions} + (j-1) \text{ multiplications} \\ &= 2 + j - 2 + j - 1 = 2j - 1 \end{aligned}$$

$$\begin{aligned} \#\text{flops for } L &= \sum_{i=1}^{n} \sum_{j=1}^{i-1} \#\text{flops for } l_{ij} = \sum_{i=1}^{n} \sum_{j=1}^{i-1} (2j-1) \\ &= \sum_{i=1}^{n} \left(2 \left(\sum_{j=1}^{i-1} j \right) - (i-1) \right) = \sum_{i=1}^{n} \left(2 \frac{(i-1)i}{2} - (i-1) \right) = \sum_{i=1}^{n} \left(i^2 - 2i \right) + n \\ &= \frac{n(n+1)(2n+1)}{6} - 2\frac{n(n+1)}{2} + n = \frac{1}{3}n^3 + \frac{1}{2}n^2 + \frac{1}{6}n - n^2 - n + n = \frac{1}{3}n^3 - \frac{1}{2}n^2 + \frac{1}{6}n \end{aligned}$$

Solve step 1:

$$\#\text{flops for } z_i = 1 \text{ subtraction} + (i-2) \text{ additions} + (i-1) \text{ multiplications} = 1 + i - 2 + i - 1 = 2i - 2$$

$$\begin{aligned}\#\text{flops for step 1} &= \sum_{i=1}^{n} \#\text{flops for } z_i = \sum_{i=1}^{n}(2i-2) \\ &= 2\left(\sum_{i=1}^{n} i\right) - 2n = 2\frac{n(n+1)}{2} - 2n = n^2 + n - 2n = n^2 - n\end{aligned}$$

Solve step 2:

$$\#\text{flops for } y_i = 1 \text{ division} = 1$$

$$\#\text{flops for step 2} = \sum_{i=1}^{n} \#\text{flops for } y_i = \sum_{i=1}^{n} 1 = n$$

Solve step 3:

$$\begin{aligned}\#\text{flops for } x_i &= 1 \text{ subtraction} + (n-i-1) \text{ additions} + (n-i) \text{ multiplications} \\ &= 1 + (n-i-1) + (n-i) = 2n - 2i\end{aligned}$$

$$\#\text{flops for step 3} = \sum_{i=1}^{n} \#\text{flops for } x_i = \sum_{i=1}^{n}(2n-2i) = 2n^2 - 2\left(\sum_{i=1}^{n} i\right) = 2n^2 - 2\frac{n(n+1)}{2} = n^2 - n$$

Summing these results, the total number of flops required to solve $Ax = b$ using the $LDL^\top$ decomposition is thus $\sim \frac{1}{3}n^3$.

8.2.2 Kernel matrix construction

In this section, we work out the computational cost in terms of flops of constructing the kernel matrix using the different considered kernel functions as described in section 2:

- Linear kernel: $K(x_1, x_2) = x_1^\top x_2$
- Polynomial kernel: $K(x_1, x_2) = (1 + c x_1^\top x_2)^d$ with $c > 0$ and polynomial degree $d \in \mathbb{N}$
- Gaussian (RBF) kernel: $K(x_1, x_2) = e^{-c\|x_1 - x_2\|_2^2}$ with $c > 0$
- Laplacian kernel: $K(x_1, x_2) = e^{-c\|x_1 - x_2\|_1}$ with $c > 0$

Due to the symmetry of the kernel matrix, we only need to explicitly compute the upper triangular part (including the diagonal)

Computing the kernel matrix:

$$\begin{aligned}\#\text{flops for } K &= \sum_{i=1}^{n} ((n-i+1)*) \\ &= \#\text{flops of a single entry} * \left(n^2 + n - \sum_{i=1}^{n}(i)\right) \\ &= \#\text{flops of a single entry} * \left(n^2 + n - \frac{n(n+1)}{2}\right) \\ &= \left(\frac{1}{2}n^2 + \frac{1}{2}n\right) * \#\text{flops of a single entry}\end{aligned}$$

Linear kernel

$$\#\text{flops for a single entry} = m \text{ multiplications} + (m-1) \text{ additions} = 2m - 1$$

$$\begin{aligned}\#\text{flops for } K &= \left(\frac{1}{2}n^2 + \frac{1}{2}n\right) * \#\text{flops of a single entry} = \left(\frac{1}{2}n^2 + \frac{1}{2}n\right) * (2m-1) \\ &= (n^2 + n)m - (\frac{1}{2}n^2 + \frac{1}{2}n)\end{aligned}$$

So, the computational cost in terms of flops is $\sim (m - \frac{1}{2})n^2$.

Polynomial kernel

$$\begin{aligned}
\#\text{flops for a single entry } &= (m+1) \text{ multiplications} + m \text{ additions} + (d-1) \text{ multiplications} = 2m + d \\
\#\text{flops for } K &= \left(\frac{1}{2}n^2 + \frac{1}{2}n\right) * \#\text{flops of a single entry} = \left(\frac{1}{2}n^2 + \frac{1}{2}n\right) * (2m + d) \\
&= (n^2 + n)m + (\frac{1}{2}n^2 + \frac{1}{2}n)d
\end{aligned}$$

So, the computational cost in terms of flops is $\sim (m + \frac{1}{2}d)n^2$.

RBF kernel

$$\begin{aligned}
\#\text{flops for a single entry } &= 1 \text{ exponential} + 1\ m\text{-dimensional squared Euclidean norm} + m \text{ subtractions} \\
&= cost_{exp} + (m \text{ multiplications} + (m-1) \text{ additions}) + m = 3m - 1 + cost_{exp} \\
\#\text{flops for } K &= \left(\frac{1}{2}n^2 + \frac{1}{2}n\right) * \#\text{flops of a single entry} = \left(\frac{1}{2}n^2 + \frac{1}{2}n\right) * (3m - 1 + cost_{exp})
\end{aligned}$$

So, the computational cost in terms of flops is $\sim (\frac{3}{2}m - \frac{1}{2} + \frac{1}{2}cost_{exp})n^2$.

Laplacian kernel

$$\begin{aligned}
\#\text{flops for a single entry } &= 1 \text{ exponential} + 1\ m\text{-dimensional 1-norm} + m \text{ subtractions} \\
&= cost_{exp} + ((m-1) \text{ additions}) + m = 2m - 1 + cost_{exp} \\
\#\text{flops for } K &= \left(\frac{1}{2}n^2 + \frac{1}{2}n\right) * \#\text{flops of a single entry} = \left(\frac{1}{2}n^2 + \frac{1}{2}n\right) * (2m - 1 + cost_{exp})
\end{aligned}$$

So, the computational cost in terms of flops is $\sim (m - \frac{1}{2} + \frac{1}{2}cost_{exp})n^2$.

8.3 Algorithms

Here, we will explain the algorithms implemented through out our report; in particular sections 5 and 6

8.3.1 NysSVD: Algorithm 5.6 from [Tropp and Webber, 2023]

Inputs:

- $C \in \mathbb{R}^{n \times s}$
- $W \in \mathbb{R}^{s \times s}$
- $S \in \mathbb{R}^{n \times s}$
- Stabilisation parameter $\varepsilon_{\text{stab}}$
  Note that we fixed $\varepsilon = 10^{-4}$ for simplicity and computational efficiency here
  [Tropp and Webber, 2023] recommends using $\varepsilon_{\text{stab}} = \varepsilon_{\text{machine}}.Tr(K)$

1. Perform stabilised Cholesky decomposition of W: $H^\top H = W + \varepsilon_{\text{stab}} S^\top S$
   Since columns are sampled without replacement, $S^\top S = I_s$
2. Compute $Z = (C + \varepsilon_{\text{stab}} S)H^{-1}$
   Since H is upper triangular, computing H^{-1} is efficient
   This step is effectively finding $CW^{-1/2}$
3. Compute the SVD of Z: $[U, \Sigma, _] = \text{svd_econ}(Z)$
   this is a 'thin' SVD, only returning the relevant columns of U, which is an orthogonal matrix
4. Remove the stabilisation: $\Lambda = \max\{0, \Sigma^2 - \varepsilon_{\text{stab}} I\}$
   This corrects any artificial inflation in the singular values introduced in step 1

Outputs:

- U, Λ such that $\hat{K} \approx U \Lambda U^\top$

8.3.2 Approximate λ-Ridge Leverage Scores from [Alaoui and Mahoney, 2015]

Inputs:

- data set X
- sampling distribution $P = (p_1, \ldots, p_n)$
- sampling parameter $p \ll n$
- KRR regularisation parameter $\lambda > 0$
- accuracy parameter $\varepsilon \in (0, \frac{1}{2})$ of our approximations (used to verify results)

1. Sample p indices $i_1, \ldots, i_p$ from $\{1, \ldots, n\}$ with replacement according to P
2. Form the sampled columns $K_{i_1}, \ldots, K_{i_p}$ of the kernel matrix using X
3. Construct matrices
   $C = [K_{i_1}, \ldots, K_{i_p}] \in \mathbb{R}^{n \times p}$
   $W \in \mathbb{R}^{p \times p}$, where $W_{ab} = K(x_{i_a}, x_{i_b})$
4. Compute the factor B such that $BB^\top = CW^\dagger C^\top$
5. for each $i \in \{1, \ldots, n\}$, compute $\tilde{\ell}_i = B_i^\top (B^\top B + n\lambda I)^{-1} B_i$, where B_i is the i-th row of B

Note: To compute B, we can first compute W using Cholesky factorisation: $W = R^\top R$ where R is an upper triangular matrix. This implies that $W^{-1} = R^{-1} R^{-\top}$. We can then write

$$\begin{aligned} BB^\top &= C(R^{-1}R^{-\top})C^\top \\ &= (CR^{-1})(R^{-\top}C^\top) \\ &= (CR^{-1})(CR^{-1})^\top, \end{aligned}$$

Giving us the result $B = CR^{-1}$.
Outputs:

- list of values of $\tilde{\ell}_i$ for $i \in \{1, \ldots, n\}$